An Anonymous Multi-Authority Key-Policy Attribute-Based Encryption Scheme With Expressive Keyword Search

Chenbin Zhao, Xiaocong Lin, Jing Chen, Weijing You, Jie Cui, *Senior Member, IEEE*, and Zhongyun Hua, *Senior Member, IEEE*

***Abstract*—Existing attribute-based searchable encryption schemes have advanced the ability to perform fine-grained keyword retrieval over encrypted data; however, they still suffer from two fundamental limitations. First, most existing constructions rely on a single trusted authority, which not only results in excessive computational burden on the centralized entity but also limits the scalability of key management in distributed environments. Second, many schemes reveal sensitive attribute or policy information in ciphertexts or search tokens, causing privacy leakage and enabling inference attacks from untrusted servers. These shortcomings highlight the need for a decentralized and privacy-preserving searchable encryption framework capable of supporting expressive queries without exposing access structures or user attributes. To overcome these limitations, we first design an anonymous Multi-Authority Key-Policy Attribute-Based Encryption (MA-KPABE) scheme. Our construction conceals the access policies embedded in users' secret keys and attribute embedded in the ciphertext, thereby limiting information leakage during key generation. Building upon this foundation, we further extend MA-KPABE into a Multi-Authority Attribute-Based Searchable Encryption (MA-ASE) scheme that integrates expressive keyword search capabilities while preserving keyword privacy. The resulting system enables decentralized key management, privacy-preserving access control, and secure expressive search within a unified cryptographic framework. We formally prove that the proposed schemes satisfy IND-CPA, anonymity, and IND-CKA security. Comprehensive performance evaluations demonstrate that the distributed architecture significantly reduces authority-side computational load and achieves practical efficiency for large-scale encrypted retrieval scenarios.**

***Index Terms*—Attribute-based encryption, keyword search, policy-hiding, multi-authority.**

Received 17 December 2025; revised 14 April 2026; accepted 29 May 2026. Date of publication 2 June 2026; date of current version 1 September 2026. This work was supported in part by the Scientific Research Foundation for High-level Talents of Anhui University of Science and Technology under Grant 2025yjrc0117, in part by the Police Integration Computing Key Laboratory of Sichuan Province under Grant JWRH202503002, in part by the Open Project of Anhui Engineering Research Center for IoT Security Technology under Grant AH-IOTS-2025-4, in part by the China Postdoctoral Science Foundation under Grant 2025M781443, in part by the Young Scientists Fund of the National Natural Science Foundation of China under Grant 62202102, in part by the the Natural Science Foundation of Fujian Province under Grant 2024J08162, and in part by the CCF-NSFOCUS 'Kunpeng' Research Fund under Grant CCF-NSFOCUS2024004. *(Corresponding author: Weijing You.)*

Chenbin Zhao is with the School of Mathematics and Big Data, Anhui University of Science and Technology, Huainan 232001, China, also with the Anhui Engineering Research Center for IoT Security Technology, Hefei 230601, China, and also with the Police Integration Computing Key Laboratory of Sichuan Province, Chengdu 610065, China (e-mail: chenbinzhao96@163.com).

Xiaocong Lin and Weijing You are with the College of Computer and Cyber Security, Fujian Normal University, Fuzhou 350117, China (e-mail: qsx20231396@student.fjnu.edu.cn; youweijing@fjnu.edu.cn).

Jing Chen is with the Key Laboratory of Aerospace Information Security and Trusted Computing, Ministry of Education, School of Cyber Science and Engineering, Wuhan University, Wuhan 430072, China (e-mail: chenjing@whu.edu.cn).

Jie Cui is with the Key Laboratory of Intelligent Computing and Signal Processing of Ministry of Education, School of Computer Science and Technology and Anhui Engineering Laboratory of IoT Security Technologies, Anhui University, Hefei 230039, China (e-mail: cuijie@mail.ustc.edu.cn).

Zhongyun Hua is with the School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen 518055, China (e-mail: huazhongyun@hit.edu.cn).

Digital Object Identifier 10.1109/TDSC.2026.3699384

I. Introduction

With the rapid growth of cloud computing and data outsourcing services, a vast amount of sensitive information is being stored and managed by third-party servers. While encryption ensures data confidentiality [1], [2], [3], it simultaneously limits the ability to perform efficient and selective data retrieval. To overcome this limitation, Asymmetric Searchable Encryption (ASE) has emerged as an effective cryptographic paradigm that enables users to perform keyword-based searches over encrypted data without revealing the plaintext to the server [4], [5], [6]. ASE has found broad applications in areas such as secure cloud storage, healthcare systems, and Internet of Things (IoT) environments. Among various ASE techniques, Attribute-Based Searchable Encryption (ABSE)—which combines the expressiveness of Attribute-Based Encryption (ABE) with keyword search functionality—provides a flexible framework for fine-grained access control and secure data retrieval in distributed systems.

Recent research on Asymmetric Searchable Encryption (ASE) has made remarkable progress by incorporating attribute-based encryption techniques to achieve both expressive search and fine-grained access control. In particular, ABE-based searchable schemes such as CWD [7] and FEASE [8] extend key-policy structures to support conjunctive and disjunctive keyword queries, enabling richer search semantics under complex access formulas. To improve efficiency in large-scale applications, several works—including FAME [9], FABEO [10], FAKS [11], and LPCR [12]—optimize encryption and search operations by refining underlying ABE primitives and

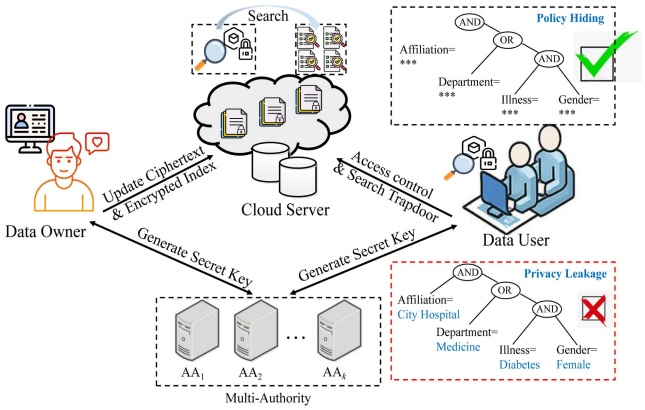


Fig. 1. System Model Framework Diagram.

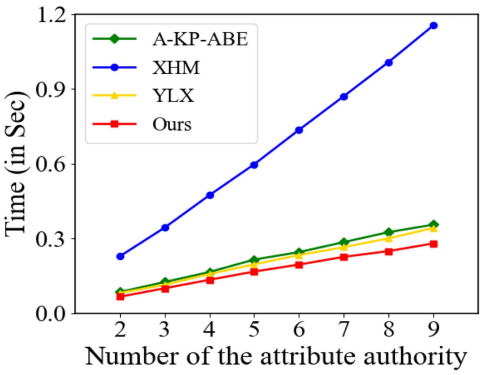


Fig. 2. Running times for AASetup.

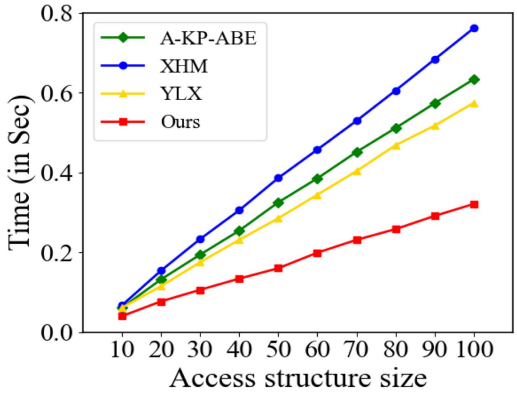


Fig. 3. Running times for Key Generation.

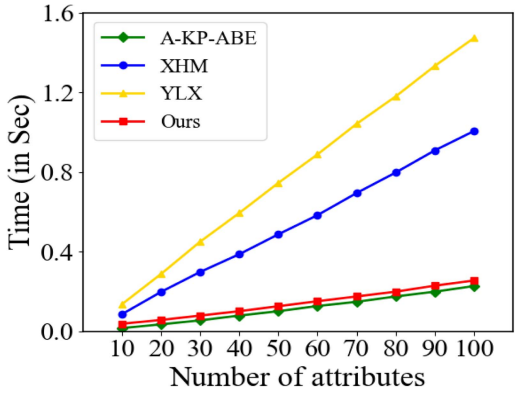


Fig. 4. Running times for Encryption.

designing lightweight index mechanisms. Beyond efficiency, enhancing privacy protection has also become a key research focus. Schemes such as those proposed by Chen et al. [5] and LPCR [12] introduce attribute or policy-hiding mechanisms to mitigate information leakage, while blockchain-assisted frameworks like Yan et al. [13] explore decentralized privacy preservation. In parallel, multi-authority ABE frameworks (e.g., Rouselakis and Waters [14], Wang et al. [15], and Xiao et al. [16]) have been developed to distribute attribute management and eliminate single points of trust. More recent efforts, such as ERS-ABE [17] and PGVMatch [18], integrate searchable encryption into multi-authority settings, marking an early step toward decentralized and privacy-aware ASE systems.

However, existing works still face several critical challenges. First, although many ABSE schemes provide expressive search and fine-grained access control [5], [11], [12], [13], [19], most of them fail to address policy or attribute privacy. In these schemes, the access structures or attribute sets are often explicitly revealed in ciphertexts or trapdoors, which may lead to sensitive information leakage about user attributes or access intentions. For instance, Chen et al. [5] achieved only partial policy hiding, while schemes such as FAKS [11], rABEKS [19], and YZB [13] do not provide any attribute-hiding capability. Second, nearly all existing ASE and PAEKS frameworks are designed under a single-authority setting, where a single trusted entity manages the system's attribute universe and key distribution. This architecture introduces a single point of trust and limits scalability in collaborative or decentralized environments. Although some multi-authority ABE schemes [14], [15], [16], [18] achieve decentralized authorization, they lack searchable functionality and do not support attribute hiding. Consequently, multi-authority ASE schemes that simultaneously ensure attribute privacy remain largely unexplored, leaving a significant research gap between secure keyword search and privacy-preserving fine-grained access control in distributed systems.

Motivated by these challenges, this paper first designs an anonymous multi-authority key-policy attribute-based encryption scheme with policy-hiding properties (MA-KPABE), which serves as the cryptographic foundation of our study. The proposed MA-KPABE enables fine-grained access control under a decentralized trust model, while simultaneously concealing access policies and attribute information from unauthorized parties, including the cloud server. Building upon this MA-KPABE construction, we further develop a multi-authority attribute-based searchable encryption (MA-ASE) scheme that supports expressive keyword search over encrypted data. By integrating efficient multi-keyword search mechanisms into the MA-KPABE framework, the proposed MA-ASE scheme bridges the gap between multi-authority ABE and searchable encryption, achieving both decentralized access control and privacy-preserving search functionality. The resulting construction ensures that neither ciphertexts nor search tokens reveal sensitive attribute-related information, while completely eliminating reliance on a centralized authority.

The main contributions are summarized as follows:

1) We design an efficient and anonymous multi-authority key-policy attribute-based encryption scheme with policy-hiding properties (MA-KPABE). Unlike existing constructions that either rely on a single authority or reveal access structures, the proposed scheme simultaneously conceals the access policies embedded in secret keys and the attribute sets associated with ciphertexts. This design prevents unauthorized entities, including the cloud server, from inferring sensitive attribute or policy information during data storage and access, while enabling fine-grained access control under a decentralized trust model.
2) Building upon the proposed MA-KPABE scheme, we further develop a novel multi-authority attribute-based searchable encryption framework with keyword policy hiding (MA-ASE). The proposed construction supports expressive multi-keyword search while eliminating the reliance on a centralized authority through a distributed key management architecture. By integrating searchable encryption mechanisms into a keyword policy-hiding encryption foundation, the scheme enables efficient ciphertext retrieval without disclosing keyword-related or attribute-related information to untrusted servers.
3) We provide rigorous security proofs for both MA-KPABE and MA-ASE under standard cryptographic models. Specifically, we show that the proposed schemes achieve: a) IND-CPA security for data confidentiality against chosen-plaintext attacks; b) anonymity against chosen access-policy attacks, ensuring the privacy of attributes and policies; and c) IND-CKA security for search token privacy against chosen-keyword attacks. In addition, extensive experimental evaluations are conducted to demonstrate the efficiency and scalability of the proposed distributed architecture, highlighting its practical advantages over existing approaches.

The remainder of this paper is organized as follows. Section II reviews related work, Section III presents the necessary preliminaries, and Section IV formulates the problem. The proposed MA-KPABE and MA-ASE schemes are described in Section V, followed by the formal security analysis in Section VI. Section VII reports the performance evaluation and comparative results, and Section VIII concludes the paper.

II. Related Work

We situate our work by reviewing the related works. This section first outlines the foundations of attribute-based encryption and its multi-authority extensions, then surveys attribute-based searchable encryption, and finally discusses these ideas to motivate our research.

1) Attribute-Based Encryption and Multi-Authority Extension: Attribute-based encryption enables fine-grained access control by associating encryption and decryption operations with descriptive attributes rather than individual identities. Initially introduced by Sahai and Waters [20], ABE later evolved into two principal paradigms: key-policy ABE (KP-ABE) [21] and ciphertext-policy ABE (CP-ABE) [22]. In KP-ABE, policies are embedded in users' secret keys, whereas in CP-ABE, they are enforced in ciphertexts. To support more expressive and scalable access control, Ostrovsky et al. [23] introduced linear secret sharing schemes to efficiently represent complex policies. Subsequent works further strengthened ABE's security [24] and efficiency [9], [10].

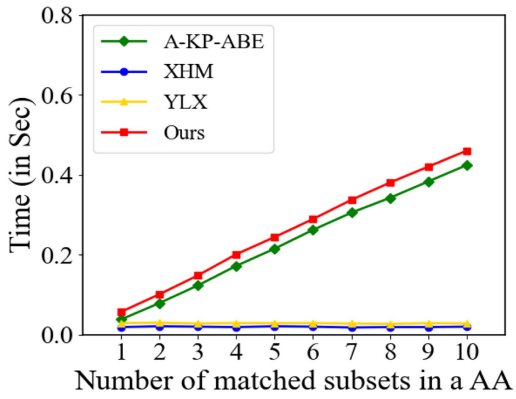


Fig. 5. Decryption with all-OR structure.

Despite these advancements, single-authority ABE schemes inherently suffer from the risk of key escrow and single-point failure. To mitigate these issues, Chase [25] introduced multi-authority ABE, enabling multiple authorities to jointly manage disjoint sets of attributes. Later improvements [14], [15], [26], [27], [28], [29], [30] enhanced privacy, efficiency, and decentralization. However, most existing works focus primarily on access control rather than ciphertext retrieval, motivating further research on searchable encryption extensions.

2) Attribute-Based Searchable Encryption: To meet the demand for secure and efficient ciphertext retrieval, ABE has been extended into attribute-based searchable encryption, which integrates keyword search into the ABE framework. This integration enables data users to retrieve encrypted data that satisfy both access policies and search keywords, significantly enhancing usability in cloud environments.

Early CP-ABE-based searchable encryption systems, such as FAKS [11], utilized bloom filter trees to improve keyword matching efficiency but relied on a single authority and did not preserve attribute privacy. Wang et al. [12] proposed a lightweight ciphertext retrieval scheme by combining searchable encryption with CP-ABE and embedding attribute values into LSSS structures to hide access policies. Zhang et al. [4] proposed an efficient ABE with keyword search supporting policy hiding, enabling secure retrieval and fine-grained access control of encrypted personal health records in cloud environments. Yan et al. [13] incorporated blockchain into a CP-ABE-based multi-keyword searchable encryption model to achieve access policy hiding in IoT environments, while Chen et al. [5] realized partial policy hiding and user revocation in a CP-ABE setting. Although privacy was enhanced, it remained a single-authority design. In addition, Ghopur et al. [31] extended CP-ABE to a decentralized multi-authority framework for e-health clouds. More recently, Li et al. [18] introduced a multi-authority attribute-based keyword

search primitive, supporting fine-grained task matching under decentralized control; however, their work does not consider attribute and policy privacy.

3) Discussion & Key Ideas: Existing ABSE schemes demonstrate significant progress in combining access control with keyword search; however, most designs [5], [11], [12], [13] either depend on a single trusted authority, limiting scalability and resilience, or fail to conceal attribute and policy information, leading to privacy leakage [11], [18], [31]. Although recent efforts have explored decentralized and partially hidden-policy constructions, an efficient and privacy-preserving multi-authority attribute-based searchable encryption framework remains insufficiently addressed. Therefore, achieving multi-authority management together with attribute and policy hiding is essential for constructing secure, privacy-preserving, and scalable public-key searchable encryption systems—particularly in distributed environments where trust decentralization and privacy protection are equally critical.

III. Preliminary

To facilitate understandings, we present preliminaries including monotone span programs, partially hidden structures, and bilinear pairings.

A. Monotone Span Programs (MSP)

Monotone Span Programs (MSP) [32] form a general class of functions that includes all monotone boolean formulas. An access structure is encoded as a triplet $\mathcal{M} = (\mathbb{F}^{\ell}, \mathbb{M}_{\ell\times n}, \pi)$, where $\mathbb{F}^{\ell}$ is a finite field, $\mathbb{M}$ is an $\ell \times n$ matrix with entries from $\mathbb{F}^{\ell}$, and π is a labeling function mapping each row to an attribute, i.e., $\pi : \{1, \ldots, \ell\} \to \mathbb{U}$.

An MSP represents a monotone boolean function $f : \{0,1\}^{|\mathbb{U}|} \to \{0,1\}$, where input variables can only change from 0 to 1. For any attribute subset $\mathbb{S} = \{u_i\}_{i\in[m]} \subseteq \mathbb{U}$, we say that $\mathbb{S}$ is *accepted* by the MSP if there exists a coefficient vector $\vec{\gamma} \in \mathbb{F}^{\ell}$ such that:

- $\vec{\gamma}^{\mathsf{T}}\mathbb{M} = \vec{e}_1 = (1, 0, \ldots, 0)$, i.e., the target vector $\vec{e}_1$ is in the linear span of the rows of $\mathbb{M}$;
- $\gamma_i = 0$ if $\pi(i) \notin S$, meaning that only the rows labeled by attributes in $\mathbb{S}$ are used in the linear combination.

In other words, if it is possible to linearly combine the matrix rows corresponding to attributes in $\mathbb{S}$ to obtain the target vector $(1, 0, \ldots, 0)$, then the set $\mathbb{S}$ is accepted, and the function $f(\mathbb{S}) = 1$.

B. Partially Hidden Structures

Partially hidden structures [8], [33] are access control mechanisms in which only attribute names are revealed, while their values remain concealed to enhance privacy. Each attribute consists of a name and a value. In these structures, attribute names are embedded in the ciphertext and secret key, forming the visible portion of the access structure and attribute set, whereas attribute values are hidden from the cloud server. During decryption, the scheme first matches attribute names and then verifies whether the hidden attribute values satisfy the access policy.

Formally, let the attribute set be denoted as $\mathbb{S} = \{u_i\}_{i\in[m]}$, where each attribute $u_i = \{n_i, v_i\}$ consists of an attribute name n_i and an attribute value v_i. Each u_i belongs to a distinct attribute category. An access policy is defined as $\mathbb{A} = \{\mathbb{M}_{\ell\times n}, \pi, \pi(i)\}$, where $\mathbb{M} \in \mathbb{F}^{\ell\times n}$ is a share-generating matrix, and $\pi : [1, \ldots, \ell] \to \mathbb{U}$ is a function that maps each row of $\mathbb{M}$ to an attribute $\pi(i) = \{n_{\pi(i)}, v_{\pi(i)}\}$. Here, $n_{\pi(i)}$ and $v_{\pi(i)}$ denote the attribute name and value of $\pi(i)$, respectively. A given attribute set $\mathbb{S}$ satisfies the policy $\mathbb{A}$ if and only if there exists a subset $\mathbb{I} \subseteq [\ell]$ and constants $\{\gamma_i \in \mathbb{F}^{\ell\times n}\}_{i\in\mathbb{I}}$ such that:

$$\sum_{i\in\mathbb{I}} \gamma_i \mathbb{M}_i = (1, 0, \ldots, 0) \quad \text{and} \quad \forall i \in \mathbb{I},\ \pi(i) = x_i,$$

where each $x_i \in \mathbb{S}$ includes both a matching attribute name and value.

In our schemes, the attribute values $v_{\pi(i)}$ from the access policy $(\mathbb{M}_{\ell\times n}, \pi, \pi(i) = \{n_{\pi(i)}, v_{\pi(i)}\})$ and the attribute values v_i in the attribute set $\mathbb{S}$ are concealed in both the ciphertext and secret key. Only the matrix $\mathbb{M}_{\ell\times n}$, the mapping function π, and the attribute names $n_{\pi(i)}$ and n_i are exposed.

C. Bilinear Pairings

In many identity/attribute-based encryption constructions, bilinear pairings serve as a fundamental tool. Let $\mathbb{G}_1$, $\mathbb{G}_2$, and $\mathbb{G}_{\mathrm{T}}$ be cyclic groups of prime order p, with generators $g_1 \in \mathbb{G}_1$ and $g_2 \in \mathbb{G}_2$, respectively. An asymmetric bilinear mapping $\hat{e} : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_{\mathrm{T}}$ satisfying the following properties:

- *Computability:* There exists an efficient polynomial-time algorithm to compute $\hat{e}(g_1, g_2) \in \mathbb{G}_{\mathrm{T}}$.
- *Bilinearity:* For all $a, b \in \mathbb{Z}_p$, $\hat{e}(g_1^a, g_2^b) = \hat{e}(g_1, g_2)^{ab}$.
- *Non-degeneracy:* There exists $g_1 \in \mathbb{G}_1$ and $g_2 \in \mathbb{G}_2$ such that $\hat{e}(g_1, g_2) \neq 1$.

IV. Problem Formulation

A. System Model

The system model consists of four entities: Cloud Server (CS), Data Owner (DO), Data User (DU), and Attribute Authority (AA), and the model framework is shown in Fig. 1.

- *Cloud Server:* Stores the encrypted data outsourced by the data owner and possesses sufficient storage and computational resources. It also processes search requests by handling trapdoors and returning the corresponding encrypted results.
- *Data Owner:* Generates and encrypts data using descriptive attributes for fine-grained access control or keywords for searchable encryption, then uploads the ciphertexts to the cloud server.
- *Data User:* Retrieves and decrypts ciphertexts. The DU uses a secret key issued by the attribute authority to decrypt attribute-encrypted data or generates trapdoors to perform keyword searches. Access is granted if the user's attributes satisfy the access policy or the keywords match.
- *Multi-Authority:* It consists of multiple independent and trusted Attribute Authorities (AAs), each responsible for

managing a disjoint subset of attributes and generating corresponding secret keys for users based on access policies. In this setting, the access policy is embedded in the user's secret key, while ciphertexts are associated with attribute sets. A user may obtain secret keys from one or more AAs, and can successfully decrypt the ciphertext only if the attribute sets associated with the ciphertext satisfy the access policy embedded in the secret key.

B. Threat Model

In the MA-KPABE and MA-ASE system, we consider potential adversarial behaviors that may threaten data confidentiality, access policy privacy, and keyword privacy. The system assumes a cloud-based outsourced storage and computation environment, where sensitive data is encrypted before outsourcing and access is controlled via attribute-based mechanisms.

The cloud server is considered honest-but-curious: it faithfully executes storage and search protocols but may attempt to infer information from ciphertexts or trapdoors, including underlying data, user attributes, access policies, or keywords. External adversaries can eavesdrop on communications between data owners, data users, and attribute authorities, aiming to intercept ciphertexts, trapdoors, or key-related messages. Malicious users may collude with others or the cloud server to access data beyond their authorization, potentially combining keys from multiple users to bypass access policies. Each attribute authority is assumed to be trusted within its domain, honestly generating secret keys for users according to access policies.

This threat model captures the realistic adversarial behavior in MA-KPABE and MA-ASE systems, ensuring that our schemes protect data confidentiality, access policy privacy, and keyword privacy against the described adversaries.

C. The Syntax of MA-KPABE

We propose an efficient and anonymous MA-KPABE scheme, which consists of five probability polynomial time algorithms: GlobalSetup, AASetup, KeyGen, Encrypt and Decrypt satisfying the following syntax.

1) $\mathsf{GlobalSetup}(1^\lambda) \to pars$: This is a global initialization algorithm executed by a trusted entity. Given a security parameter $\lambda \in \mathbb{N}$, it generates and outputs the global system parameters $pars$, which are shared across all entities in the system.
2) $\mathsf{AASetup}(pars, k) \to \{PK_k, MK_k\}$: Each attribute authority executes this algorithm to generate its key pair. It takes as input the global parameters $pars$ and the authority's index k, and outputs a public key PK_k and a mastersecret key MK_k for the k-th attribute authority.
3) $\mathsf{KeyGen}(pars, id_u, MK_k, \mathbb{A}_u) \to SK_u$: This algorithm is run by the attribute authority to generate a user's decryption key. It takes as input the global parameters $pars$, the identity id_u of the data user, the authority's master secret key MK_k, and an access structure $\mathbb{A}_u$ representing the user's access policy. It outputs the user's secret key SK_u.
4) $\mathsf{Encrypt}(pars, msg, PK_k, \mathbb{S}) \to CT$: This encryption algorithm is executed by the data owner. It takes as input the global public parameters $pars$, a plaintext message msg, a public key PK_k from the attribute authority, and a set of attributes $\mathbb{S}$. It outputs the ciphertext CT associated with $\mathbb{S}$.
5) $\mathsf{Decrypt}(SK_u, CT) \to msg/\perp$: This algorithm is executed by the data user to recover the message. It takes as input the user's secret key SK_u, which encodes an access structure $\mathbb{A}_u$, and a ciphertext CT associated with an attribute set $\mathbb{S}$. If the set $\mathbb{S}$ satisfies the access structure $\mathbb{A}_u$, the algorithm outputs the plaintext message msg; otherwise, it returns $\perp$ indicating decryption failure.

Correctness: For any security parameter λ, any message msg, any attribute set $\mathbb{S}$, and any access structure $\mathbb{A}_u$, let

$$
\begin{aligned}
&pars \leftarrow \mathsf{GlobalSetup}(1^\lambda),\\
&(PK_k, MK_k) \leftarrow \mathsf{AASetup}(pars, k),\\
&SK_u \leftarrow \mathsf{KeyGen}(pars, id_u, MK_k, \mathbb{A}_u),\\
&CT \leftarrow \mathsf{Encrypt}(pars, msg, PK_k, \mathbb{S}).
\end{aligned}
$$

The MA-KPABE scheme is *correct* if for all such inputs,

$$\Pr\left[\, \mathsf{Decrypt}(SK_u, CT) = msg \;\middle|\; \mathbb{S} \models \mathbb{A}_u \,\right] = 1.$$

D. The Syntax of MA-ASE

We propose a fast and expressive MA-ASE scheme, which consists of five probability polynomial time algorithms: GlobalSetup, AASetup, TrapGen, Encrypt, and Search satisfying the following syntax.

1) $\mathsf{GlobalSetup}(1^\lambda) \to pars$: This algorithm is executed to initialize the system. It takes as input the security parameter $\lambda \in \mathbb{N}$ and outputs the global public parameters $pars$.
2) $\mathsf{AASetup}(pars, k) \to \{PK_k, MK_k\}$: This algorithm is executed by each attribute authority to generate its key pair. It takes as input the global public parameters pars and the total number of attribute authorities k. It outputs a public key PK_k and a master secret key MK_k for the k-th attribute authority.
3) $\mathsf{TrapGen}(pars, id_u, MK_k, \mathbb{K}_u) \to TD_u$: This algorithm is executed by the attribute authority to generate a trapdoor for a data user. It takes as input the global public parameters $pars$, the identifier of the data user id_u, the master secret key MK_k, and the user's keyword access policy $\mathbb{K}_u$. It outputs a trapdoor TD_u corresponding to the specified policy.
4) $\mathsf{Encrypt}(pars, PK_k, \mathbb{W}) \to CT$: This algorithm is executed by the data owner to encrypt a set of keywords. It takes as input the global public parameters $pars$, the public key PK_k of the AA, and a set of keywords $\mathbb{W}$. It outputs the resulting ciphertext CT.
5) $\mathsf{Search}(CT, TD_u) \to 1/0$: This algorithm is executed by the cloud server to perform a search. It takes as input a ciphertext CT associated with a keyword set $\mathbb{W}$, and a trapdoor TD_u associated with a keyword access policy

$\mathbb{P}_u$. It outputs 1 if the keyword set satisfies the access policy (i.e., the search is successful), or 0 otherwise.

Correctness: For any security parameter λ, any keyword set $\mathbb{W}$, and any keyword access policy $\mathbb{K}_u$, let

$$pars \leftarrow \mathsf{GlobalSetup}(1^\lambda),$$
$$(PK_k, MK_k) \leftarrow \mathsf{AASetup}(pars, k),$$
$$TD_u \leftarrow \mathsf{TrapGen}(pars, id_u, MK_k, \mathbb{K}_u),$$
$$CT \leftarrow \mathsf{Encrypt}(pars, PK_k, \mathbb{W}).$$

The MA-ASE is said to be *correct* if for all such inputs,

$$\Pr\left[\mathsf{Search}(CT, TD_u) = 1 \;\middle|\; \mathbb{W} \models \mathbb{K}_u\right] = 1.$$

E. Security Definitions

In this section, we present the formal security definitions and models for the multi-authority key-policy attribute-based encryption (MA-KPABE) and the multi-authority attribute-based searchable encryption (MA-ASE) schemes.

1) Security Definitions of MA-KPABE: The proposed MA-KPABE scheme with a partially hidden structure is defined by two fundamental security properties. First, the ciphertext must not leak any information about the underlying plaintext message, a property known as *Indistinguishability against Chosen Plaintext Attacks (IND-CPA)*. Second, the ciphertext must hide all information about the associated attribute set, a property referred to as *Anonymity*.

Definition 1 (IND-CPA Security): An MA-KPABE scheme achieves IND-CPA security if no probabilistic polynomial-time (PPT) adversary can distinguish between the encryptions of two equal-length messages under a chosen attribute set, except with negligible advantage.

The IND-CPA security is modeled through the following game between a PPT adversary $\mathcal{A}$ and a challenger $\mathcal{C}$:

- *Setup*: $\mathcal{C}$ executes $\mathsf{AASetup}(1^\lambda, k)$ to generate a public key PK_k and a master secret key MK_k. It provides PK_k to $\mathcal{A}$ while keeping MK_k private.
- *Phase 1*: $\mathcal{A}$ adaptively makes a polynomial number of queries to a key generation oracle. Given an access structure $\mathbb{A}$, the oracle computes $SK \leftarrow \mathsf{KeyGen}(pars, id, MK_k, \mathbb{A})$ and returns SK to $\mathcal{A}$.
- *Challenge*: $\mathcal{A}$ outputs a challenge attribute set $\mathbb{S}^*$ and two equal-length messages (msg_0^*, msg_1^*), subject to the restriction that $\mathbb{S}^*$ does not satisfy any access structure queried in Phase 1. $\mathcal{C}$ selects a random bit $b \in \{0,1\}$, computes $CT_b^* \leftarrow \mathsf{Enc}(pars, msg_b^*, PK_k, \mathbb{S}^*)$, and returns CT_b^* to $\mathcal{A}$.
- *Phase 2*: $\mathcal{A}$ continues to query the key generation oracle with the restriction that no queried access structure $\mathbb{A}$ can be satisfied by $\mathbb{S}^*$.
- *Guess*: $\mathcal{A}$ outputs $b' \in \{0,1\}$ and wins if $b' = b$.

An MA-KPABE scheme is adaptively IND-CPA secure if the adversary's advantage

$$\mathrm{Adv}_{\mathcal{A}}^{\mathsf{IND-CPA}}(\lambda) = \left|\Pr[b' = b] - \tfrac{1}{2}\right|$$

is negligible in the security parameter λ.

Definition 2 (Anonymity): An MA-KPABE scheme achieves anonymity if no probabilistic polynomial-time adversary can obtain any non-negligible advantage in distinguishing attribute sets associated with ciphertexts.

The anonymity property is defined through the following game between $\mathcal{A}$ and $\mathcal{C}$:

- *Setup*: $\mathcal{C}$ runs $\mathsf{AASetup}(1^\lambda, k)$ to generate PK_k and MK_k. It sends PK_k to $\mathcal{A}$ while keeping MK_k private.
- *Phase 1*: $\mathcal{A}$ adaptively queries the key generation oracle as described above.
- *Challenge*: $\mathcal{A}$ submits a challenge message msg^* and two equal-sized attribute sets $\mathbb{S}_0^* = \{n_i, v_{i0}\}_{i\in[m]}$ and $\mathbb{S}_1^* = \{n_i, v_{i1}\}_{i\in[m]}$, such that they share identical attribute names $\{n_i\}_{i\in[m]}$ and neither satisfies any queried access structure. $\mathcal{C}$ randomly selects $b \in \{0,1\}$, computes $CT_b^* \leftarrow \mathsf{Enc}(pars, msg^*, PK_k, \mathbb{S}_b^*)$, and returns CT_b^*.
- *Phase 2*: Same as Phase 1, with the restriction that no queried access structure can be satisfied by $\mathbb{S}_0^*$ or $\mathbb{S}_1^*$.
- *Guess*: $\mathcal{A}$ outputs b' and wins if $b' = b$.

An MA-KPABE scheme is anonymous if the adversary's advantage

$$\mathrm{Adv}_{\mathcal{A}}^{\mathrm{Anon}}(\lambda) = \left|\Pr[b' = b] - \tfrac{1}{2}\right|$$

is negligible in the security parameter λ.

2) Security Definition of MA-ASE: The proposed MA-ASE scheme with a partially hidden structure mandates that ciphertexts must not leak any information about the underlying keyword values. This property is formally defined as *Indistinguishability against Chosen Keyword Attacks (IND-CKA)*.

Definition 3 (IND-CKA Security): An MA-ASE scheme is said to achieve IND-CKA security if no probabilistic polynomial-time adversary can distinguish between the encryptions of two equal-length keyword sets under a chosen attribute set, except with negligible advantage.

The IND-CKA security is defined via the following game between a PPT adversary $\mathcal{A}$ and a challenger $\mathcal{C}$:

- *Setup*: $\mathcal{C}$ executes $\mathsf{AASetup}(1^\lambda, k)$ to generate the public key and master secret key (PK_k, MK_k), where PK_k is given to $\mathcal{A}$ and MK_k is kept private.
- *Phase 1*: $\mathcal{A}$ adaptively makes a polynomial number of queries to a trapdoor oracle. Given a keyword policy $\mathbb{K}$, the oracle computes $TD \leftarrow \mathsf{TrapGen}(pars, id, MK_k, \mathbb{K})$ and returns TD.
- *Challenge*: $\mathcal{A}$ outputs two equal-sized keyword sets $\mathbb{W}_0^* = \{n_i, v_{i0}\}_{i\in[m]}$ and $\mathbb{W}_1^* = \{n_i, v_{i1}\}_{i\in[m]}$, such that they share identical keyword names $\{n_i\}_{i\in[m]}$ and neither satisfies any previously queried trapdoor. $\mathcal{C}$ selects $b \in \{0,1\}$, computes $CT_b^* \leftarrow \mathsf{Enc}(pars, PK_k, \mathbb{W}_b^*)$, and returns CT_b^*.
- *Phase 2*: $\mathcal{A}$ continues querying the trapdoor oracle with the restriction that no queried keyword policy $\mathbb{K}$ can be satisfied by $\mathbb{W}_0^*$ or $\mathbb{W}_1^*$.
- *Guess*: $\mathcal{A}$ outputs b' and succeeds if $b' = b$.

An MA-ASE scheme is adaptively IND-CKA secure if

$$\mathrm{Adv}_{\mathcal{A}}^{\mathsf{IND-CKA}}(\lambda) = \left|\Pr[b' = b] - \tfrac{1}{2}\right|$$

is negligible in λ for any PPT adversary.

V. Our Proposed Schemes

We first introduce the concrete construction of our efficient and anonymous MA-KPABE scheme, and then propose the concrete construction of fast and expressive MA-ASE scheme.

A. Concrete Construction of MA-KPABE

Let $\mathbb{S} = \{u_i\}_{i\in[m]}$ denote an attribute set, where each attribute $u_i = \{n_i, v_i\}$ consists of an attribute name n_i and an attribute value v_i. The access structure is defined as $\mathbb{A} = (\mathbb{M}, \pi, \pi(i))$, where $\mathbb{M}$ is an $\ell \times n$ sharing matrix, $\mathbb{M}_i$ denotes the i-th row of the matrix, and π is a mapping function from $\mathbb{M}_i$ to an attribute $\pi(i)$. Each attribute $\pi(i) = \{n_{\pi(i)}, v_{\pi(i)}\}$ corresponds to an attribute name and an attribute value. In our construction, attribute values are hidden in both ciphertexts and secret keys to enhance privacy. The access structure follows a linear secret sharing scheme, where the matrix $\mathbb{M}$ is used to distribute a secret across multiple attributes. Only when a valid attribute set satisfies the policy, the secret can be reconstructed via linear combinations of the corresponding rows.

1) $\mathsf{GlobalSetup}(1^\lambda) \to pars$: This algorithm initializes the global system parameters. Let $\mathbb{G}_1, \mathbb{G}_2$, and $\mathbb{G}_\mathrm{T}$ be multiplicative cyclic groups of prime order p, with generators g_1 and g_2 for $\mathbb{G}_1$ and $\mathbb{G}_2$, respectively. Define a bilinear pairing $\hat{e} : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_\mathrm{T}$. Let $H : \{0,1\}^* \to \mathbb{G}_1$ and $H_1 : \{0,1\}^* \to \mathbb{Z}_p^*$ be cryptographic hash functions. The algorithm outputs the system public parameters:

$$pars = (\hat{e}, p, g_1, g_2, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_\mathrm{T}, H, H_1).$$

2) $\mathsf{AASetup}(pars, k) \to \{PK_k, MK_k\}$: Each attribute authority AA_k independently generates its public and master keys. It selects random values $\alpha_k, \beta_k^1, \beta_k^2 \in \mathbb{Z}_p$ and computes $g_2^{\beta_k^1}, g_2^{\beta_k^2}$, and $\hat{e}(g_1, g_2)^{\alpha_k}$. Each attribute authority operates independently, ensuring a distributed trust model. The secret parameter α_k is later embedded into ciphertexts, while β_k^1 and β_k^2 are used to randomize key components. Finally, AA_k publishes the public key PK_k and keeps the master key MK_k:

$$PK_k = \left(g_2^{\beta_k^1}, g_2^{\beta_k^2}, e(g_1, g_2)^{\alpha_k}\right), MK_k = \left(\beta_k^1, \beta_k^2, \alpha_k\right).$$

3) $\mathsf{KeyGen}(pars, id_u, MK_k, \mathbb{A}_u) \to SK_u$: This algorithm is executed by the k-th attribute authority AA_k to generate a decryption key for user u. AA_k randomly selects an $(n-1)$-dimensional vector $\vec{v}_k \in \mathbb{Z}_p^{n-1}$. Given the access structure $\mathbb{A}_u^k = (\mathbb{M}_u^k, \pi_u^k, \pi_u^k(i)_{i\in[\ell]})$ associated with the user, where $\mathbb{M}_u^k \in \mathbb{Z}_p^{\ell\times n}$ is a linear secret-sharing matrix and $\mathbb{M}_{u,i}^k$ denotes its i-th row vector. The authority computes:

$$sk_{u,1}^k = g_2^{H_1(id_u)\alpha_k},$$

$$sk_{u,2}^{k,i} = \left(g_1^{\mathbb{M}_{u,i}^k} \cdot (\alpha_k || \vec{v}_k)^\top \cdot H(\pi_u^k(i))^{H_1(id_u)\cdot\alpha_k}\right)^{\frac{1}{\beta_k^1}},$$

$$sk_{u,3}^{k,i} = \left(g_1^{\mathbb{M}_{u,i}^k} \cdot (\alpha_k || \vec{v}_k)^\top \cdot H(\pi_u^k(i))^{H_1(id_u)\cdot\alpha_k}\right)^{\frac{1}{\beta_k^2}}.$$

To further protect privacy, the access structure is concealed by replacing attribute values with attribute names, yielding a hidden structure $\overline{\mathbb{A}_u^k} = (\mathbb{M}_u^k, \pi_u^k, \{n_{\pi_u^k(i)}\}_{i\in[\ell]})$. The partial secret key issued by AA_k is then: $SK_u^k = (\overline{\mathbb{A}_u^k}, sk_{u,1}^k, \{sk_{u,2}^{k,i}, sk_{u,3}^{k,i}\}_{i\in[\ell]})$. Finally, the complete decryption key for user u is obtained as the union of all partial keys: $SK_u = \{SK_u^k\}_{k\in[K]}$.

4) $\mathsf{Encrypt}(pars, msg, PK_k, \mathbb{S}) \to CT$: The data owner encrypts a message msg under the attribute set $\mathbb{S} = \{u_i\}_{i\in[m]}$. It first selects two random values $s_1, s_2 \in \mathbb{Z}_p$ and sets $s = s_1 + s_2 \in \mathbb{Z}_p$. This encryption design splits randomness into two independent components s_1 and s_2, which improves security by preventing direct recovery of the global secret exponent s. The following values are then computed:

$$ct_{1,i} = H(u_i)^s, ct_{2,k} = g_2^{\beta_k^1 \cdot s_1},$$

$$ct_{3,k} = g_2^{\beta_k^2 \cdot s_2}, ct_4 = msg \cdot \prod_{k\in\Gamma} \hat{e}(g_1, g_2)^{\alpha_k s},$$

where $\Gamma \subseteq [K]$ denotes the subset of attribute authorities involved in the encryption. The term $ct_{1,i}$ binds each attribute to the randomness s, ensuring only matching attributes can contribute to successful decryption. The component ct_4 hides the plaintext using pairing-based masking, which can only be removed if the correct secret is reconstructed.

To ensure privacy, the ciphertext conceals actual attribute values. The final ciphertext is structured as:

$$CT = \left(\{n_i\}_{i\in[m]}, \{ct_{1,i}\}_{i\in[m]}, \{ct_{2,k}, ct_{3,k}\}_{k\in[\Gamma]}, ct_4\right),$$

where $\{n_i\}$ are the obfuscated attribute names corresponding to the encrypted attributes.

5) $\mathsf{Decrypt}(SK_u, CT) \to msg/\perp$: A data user employs their decryption key SK_u to recover the original message. For each $k \in [K]$, the user checks whether there exists a subset $\mathbb{I}_k \subseteq [m]$ such that the hidden attribute names $\{n_{\pi(i)}\}_{i\in[\ell]}$ in $\overline{\mathbb{A}_u^k}$ match a subset of $\{n_i\}$ in the ciphertext. If no such subset exists, output $\perp$. Otherwise, the user finds constants $\{y_{u,i}^k \in \mathbb{Z}_p\}_{i\in\mathbb{I}_k}$ such that: $\{\sum_{i\in\mathbb{I}_k} y_{u,i}^k \cdot \mathbb{M}_{u,i}^k = (1, 0, \ldots, 0)\}_{k\in\Gamma}$.

Using this relation, the user computes:

$$E_1^k = \hat{e}\left(\prod_{i\in\mathbb{I}_k} (ct_{1,\pi_u^k(i)})^{y_{u,i}^k}, sk_{u,1}^k\right),$$

$$E_2^k = \hat{e}\left(\prod_{i\in\mathbb{I}_k} \left(sk_{u,2}^{k,i}\right)^{y_{u,i}^k}, ct_2\right),$$

$$E_3^k = \hat{e}\left(\prod_{i\in\mathbb{I}_k} \left(sk_{u,3}^{k,i}\right)^{y_{u,i}^k}, ct_3\right).$$

Finally, the original message msg is recovered as:

$$msg = \prod_{k\in\Gamma} \frac{E_1^k \cdot ct_4}{E_2^k \cdot E_3^k}.$$

If the decryption is successful (i.e., the equation holds), output msg. Otherwise, proceed to the next matching subset $\mathbb{I}_k \in \mathbb{S}$. If all attempts fail, output $\perp$.

Correctness: We prove that the MA-KPABE scheme correctly recovers the plaintext when the attribute set satisfies the access structure. If the attribute set $\mathbb{S}$ satisfies the access structure $\mathbb{A}_u^k$,

there exists a subset $\mathbb{I}_k \subseteq [m]$ and constants $\{y_{u,i}^k \in \mathbb{Z}_p\}_{i\in\mathbb{I}_k}$ such that: $\sum_{i\in\mathbb{I}_k} y_{u,i}^k \cdot \mathbb{M}_{u,i}^k = (1, 0, \ldots, 0)$.

According to the construction, by substituting $ct_{1,i} = H(u_i)^s$ and $sk_{u,1}^k = g_2^{H_1(id_u)\alpha_k}$ into the above equation, we obtain:

$$E_1^k = \hat{e}\left(\prod_{i\in\mathbb{I}_k} (ct_{1,\pi_u^k(i)})^{y_{u,i}^k}, sk_{u,1}^k\right) = \hat{e}\left(\prod_{i\in\mathbb{I}_k} H\left(\pi_u^k(i)\right)^{s\cdot y_{u,i}^k}, g_2^{H_1(id_u)\alpha_k}\right).$$

Similarly, for E_2^k and E_3^k, by substituting the corresponding expressions and using bilinearity, we have:

$$E_2^k \cdot E_3^k = \hat{e}(g_1, g_2)^{\alpha_k\cdot(s_1+s_2)} \cdot \hat{e}\left(\prod_{i\in\mathbb{I}_k} H\left(\pi_u^k(i)\right)^{s\cdot y_{u,i}^k}, g_2^{H_1(id_u)\alpha_k}\right),$$

$$= \hat{e}(g_1, g_2)^{\alpha_k\cdot s} \cdot \hat{e}\left(\prod_{i\in\mathbb{I}_k} H\left(\pi_u^k(i)\right)^{s\cdot y_{u,i}^k}, g_2^{H_1(id_u)\alpha_k}\right)$$

It can be observed that the hash-related pairing terms in E_1^k cancel out with those in $E_2^k \cdot E_3^k$. Therefore, we obtain:

$$\frac{E_1^k}{E_2^k \cdot E_3^k} = \hat{e}(g_1, g_2)^{-\alpha_k s}.$$

Finally, combining with $ct_4 = msg \cdot \prod_{k\in\Gamma} \hat{e}(g_1, g_2)^{\alpha_k s}$, we have:

$$\prod_{k\in\Gamma} \frac{E_1^k \cdot ct_4}{E_2^k \cdot E_3^k} = msg.$$

Thus, when the access structure is satisfied, the decryption algorithm correctly recovers the original message msg. Otherwise, no valid reconstruction coefficients exist, and the decryption fails, outputting $\perp$. □

B. The Concrete Construction of MA-ASE

This section presents the concrete construction of the MA-ASE scheme, which can be viewed as a transformation from attribute-based encryption to a public-key searchable encryption framework, where access structures are mapped into keyword policies. Specifically, our design establishes a unified representation by transforming the access structure $\mathbb{A}_u$ and attribute set $\mathbb{S}$ in MA-KPABE into the keyword policy $\mathbb{K}_u$ and keyword set $\mathbb{W}$, respectively. The rationale behind this transformation lies in the inherent structural consistency between attributes and keywords, where both can be represented as name–value pairs, enabling a natural and lossless mapping and allowing the direct reuse of the linear secret-sharing access structure $(\mathbb{M}, \pi)$ without introducing additional policy construction overhead. Furthermore, by embedding the access policy into the trapdoor (analogous to the secret key in KPABE) and associating ciphertexts with keyword sets, the proposed MA-ASE scheme inherits the expressiveness of MA-KPABE, thereby supporting fine-grained and expressive keyword search while preserving the privacy of keyword values and access structures. In this way, the transformation tightly couples search functionality with access control, resulting in a unified and systematic extension from MA-KPABE to MA-ASE that simplifies the overall design while enhancing both expressiveness and privacy guarantees.

The $\mathbb{W} = \{u_i\}_{i\in[m]}$ is a keyword set, where each keyword $u_i = \{n_i, v_i\}$ consists of a keyword name n_i and a keyword value v_i. Similarly, the keyword policy $\mathbb{K} = (\mathbb{M}, \pi, \pi(i))$ contains three components: $\mathbb{M}$ is an $\ell \times n$ sharing matrix, $\mathbb{M}_i$ denotes its i-th row, and π is a mapping function from $\mathbb{M}_i$ to a keyword $\pi(i)$. Each keyword $\pi(i) = \{n_{\pi(i)}, v_{\pi(i)}\}$ contains a keyword name and a keyword value. To enhance privacy, keyword values are not exposed in ciphertexts or trapdoors. The keyword policy follows a linear secret sharing scheme, where the matrix $\mathbb{M}$ is used to distribute a secret across multiple keywords. Only when a valid keyword set satisfies the policy, the search operation can be successfully executed.

1) $\mathsf{GlobalSetup}(1^\lambda) \to pars$: This algorithm is consistent with the global setup of the MA-KPABE scheme. Let $\mathbb{G}_1$, $\mathbb{G}_2$, and $\mathbb{G}_\mathrm{T}$ be multiplicative cyclic groups of prime order p, with generators g_1 and g_2 for $\mathbb{G}_1$ and $\mathbb{G}_2$, respectively. Let $\hat{e} : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_\mathrm{T}$ be a bilinear map. Additionally, define two hash functions: $H : \{0,1\}^* \to \mathbb{G}_1$ and $H_1 : \{0,1\}^* \to \mathbb{Z}_p^*$. The algorithm outputs the global public parameters:

$$pars = (\hat{e}, p, g_1, g_2, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_\mathrm{T}, H, H_1).$$

2) $\mathsf{AASetup}(pars, k) \to \{PK_k, MK_k\}$: Each attribute authority AA_k independently generates its key pair. It selects random values $\alpha_k, \beta_k^1, \beta_k^2 \in \mathbb{Z}_p$ and computes $g_2^{\beta_k^1}$, $g_2^{\beta_k^2}$, and $\hat{e}(g_1, g_2)^{\alpha_k}$. The authority then publishes the public key PK_k and keeps the master key MK_k secret:

$$PK_k = (g_2^{\beta_k^1}, g_2^{\beta_k^2}, \hat{e}(g_1, g_2)^{\alpha_k}), \quad MK_k = (\beta_k^1, \beta_k^2, \alpha_k).$$

3) $\mathsf{TrapGen}(pars, id_u, MK_k, \mathbb{K}_u) \to TD_u$: This algorithm mirrors the KeyGen procedure in the ABE setting, but generates a search trapdoor based on a keyword policy instead of a decryption key based on an access structure. This reflects the shift from attribute-based encryption to searchable encryption.

Each AA_k selects a random $(n-1)$-dimensional vector $\vec{v}_k \in \mathbb{Z}_p^{n-1}$. Given the keyword policy $\mathbb{K}_u^k = (\mathbb{M}_u^k, \pi_u^k, \{\pi_u^k(i)\}_{i\in[\ell]})$, it computes:

$$td_{u,1}^k = g_2^{H_1(id_u)\alpha_k},$$

$$td_{u,2}^{k,i} = \left(g_1^{\mathbb{M}_{u,i}^k} \cdot (\alpha_k || \vec{v}_k)^\top \cdot H(\pi_u^k(i))^{H_1(id_u)\cdot\alpha_k}\right)^{1/\beta_k^1},$$

$$td_{u,3}^{k,i} = \left(g_1^{\mathbb{M}_{u,i}^k} \cdot (\alpha_k || \vec{v}_k)^\top \cdot H(\pi_u^k(i))^{H_1(id_u)\cdot\alpha_k}\right)^{1/\beta_k^2}.$$

To preserve privacy, the policy is obfuscated by replacing keyword values with keyword names, yielding $\overline{\mathbb{K}_u^k} = (\mathbb{M}_u^k, \pi_u^k, \{n_{\pi(i)}\}_{i\in[\ell]})$. Thus, the partial trapdoor generated by AA_k is:

$$TD_u^k = \left(\overline{\mathbb{K}_u^k}, td_{u,1}^k, \{td_{u,2}^{k,i}, td_{u,3}^{k,i}\}_{i\in[\ell]}\right).$$

The final trapdoor for user u is the collection of all partial trapdoors:

$$TD_u = \{TD_u^k\}_{k\in[K]}.$$

4) $\mathsf{Encrypt}(pars, PK_k, \mathbb{W}) \to CT$: The data owner chooses $s_1, s_2 \in \mathbb{Z}_p$ at random, sets $s = s_1 + s_2$, and computes:

$$ct_{1,i} = H(u_i)^s, \quad ct_{2,k} = g_2^{\beta_k^1 \cdot s_1},$$
$$ct_{3,k} = g_2^{\beta_k^2 \cdot s_2}, \quad ct_4 = \prod_{k\in\Gamma} \hat{e}(g_1, g_2)^{\alpha_k s},$$

where $\Gamma \subseteq [K]$ is the set of attribute authorities involved in the encryption. This encryption design splits randomness into two independent components s_1 and s_2, which improves security by preventing direct recovery of the global secret exponent s. The ciphertext conceals keyword values, and is defined as:

$$CT = \left(\{n_i\}_{i\in[m]}, \{ct_{1,i}\}_{i\in[m]}, \{ct_{2,k}, ct_{3,k}\}_{k\in[\Gamma]}, ct_4\right).$$

5) $\mathsf{Search}(CT, TD_u) \to 1/0$: Given a ciphertext CT and trapdoor TD_u, the cloud server determines whether there exists a subset $\mathbb{I}_k \subseteq \mathbb{W}$ such that the keyword names $\{n_i\}_{i\in[m]}$ in CT match the names $\{n_{\pi(i)}\}_{i\in[\ell]}$ in $\overline{\mathbb{K}_u^k}$. If no such subset exists, output 0. Otherwise, find constants $\{y_{u,i}^k \in \mathbb{Z}_p\}_{i\in\mathbb{I}_k}$ such that $\{\sum_{i\in\mathbb{I}_k} y_{u,i}^k \cdot \mathbb{M}_{u,i}^k = (1, 0, \ldots, 0)\}_{k\in\Gamma}$, and compute:

$$E_1^k = \hat{e}\left(\prod_{i\in\mathbb{I}_k} (ct_{1,\pi_u^k(i)})^{y_{u,i}^k}, td_{u,1}^k\right),$$
$$E_2^k = \hat{e}\left(\prod_{i\in\mathbb{I}_k} \left(td_{u,2}^{k,i}\right)^{y_{u,i}^k}, ct_2\right),$$
$$E_3^k = \hat{e}\left(\prod_{i\in\mathbb{I}_k} \left(td_{u,3}^{k,i}\right)^{y_{u,i}^k}, ct_3\right).$$

Finally, verify the correctness of the search by checking:

$$ct_4 = \prod_{k\in\Gamma} \frac{E_2^k \cdot E_3^k}{E_1^k}.$$

If the equation holds, return 1. Otherwise, continue with another subset $\mathbb{I}_k \subseteq \mathbb{W}$. If no subset satisfies the condition, return 0.

Correctness: We prove that the proposed MA-ASE scheme correctly returns 1 if and only if the keyword set $\mathbb{W}$ satisfies the keyword policy $\mathbb{K}_u^k$. If $\mathbb{W}$ satisfies $\mathbb{K}_u^k$, there exists a subset $\mathbb{I}_k \subseteq [m]$ and constants $\{y_{u,i}^k \in \mathbb{Z}_p\}_{i\in\mathbb{I}_k}$ such that: $\sum_{i\in\mathbb{I}_k} y_{u,i}^k \cdot \mathbb{M}_{u,i}^k = (1, 0, \ldots, 0)$.

According to the construction, by substituting $ct_{1,i} = H(u_i)^s$ and $td_{u,1}^k = g_2^{H_1(id_u)\alpha_k}$, we have:

$$E_1^k = \hat{e}\left(\prod_{i\in\mathbb{I}_k} (ct_{1,\pi_u^k(i)})^{y_{u,i}^k}, td_{u,1}^k\right)$$
$$= \hat{e}\left(\prod_{i\in\mathbb{I}_k} H\left(\pi_u^k(i)\right)^{s\cdot y_{u,i}^k}, g_2^{H_1(id_u)\alpha_k}\right).$$

Similarly, for E_2^k and E_3^k, by substituting the trapdoor components and ciphertext components and applying bilinearity, we obtain:

$$E_2^k \cdot E_3^k = \hat{e}(g_1, g_2)^{\alpha_k(s_1+s_2)}$$
$$\cdot \hat{e}\left(\prod_{i\in\mathbb{I}_k} H\left(\pi_u^k(i)\right)^{s\cdot y_{u,i}^k}, g_2^{H_1(id_u)\alpha_k}\right)$$
$$= \hat{e}(g_1, g_2)^{\alpha_k s} \cdot \hat{e}\left(\prod_{i\in\mathbb{I}_k} H\left(\pi_u^k(i)\right)^{s\cdot y_{u,i}^k}, g_2^{H_1(id_u)\alpha_k}\right).$$

It follows that the hash-related pairing terms in E_1^k cancel out with those in $E_2^k \cdot E_3^k$, yielding:

$$\frac{E_2^k \cdot E_3^k}{E_1^k} = \hat{e}(g_1, g_2)^{\alpha_k s}.$$

Finally, combining with $ct_4 = \prod_{k\in\Gamma} \hat{e}(g_1, g_2)^{\alpha_k s}$, we obtain:

$$\prod_{k\in\Gamma} \frac{E_2^k \cdot E_3^k}{E_1^k} = ct_4.$$

Therefore, the verification equation holds and the algorithm outputs 1. Otherwise, if the keyword set does not satisfy the policy, no valid reconstruction coefficients exist, and the equation does not hold, resulting in output 0. □

VI. Security Analysis

In this section, we analyze the security of the proposed MA-KPABE and MA-ASE schemes in the Generic Group Model (GGM) [8], where all hash functions are modeled as random oracles. The generic group model provides an abstraction in which elements of the underlying bilinear groups are represented as random encodings, and all group operations are performed through oracles. This abstraction prevents adversaries from exploiting any specific representation of group elements and restricts them to generic algebraic manipulations. As a result, the GGM captures the hardness of deriving non-trivial algebraic relations among group elements in a general and implementation-independent manner.

The rationale for adopting the GGM in our work is threefold. First, the GGM is widely used in the security analysis of pairing-based cryptographic constructions, especially in attribute-based encryption schemes [10], and is considered a standard and well-accepted framework. Second, our constructions follow the algebraic structure of efficient ABE schemes, where security fundamentally relies on the infeasibility of reconstructing hidden linear relations among exponents without satisfying the corresponding access structure. In such settings, the GGM provides a natural and effective tool for formal analysis. Third, the GGM can be viewed as a unifying abstraction underlying classical hardness assumptions in bilinear groups, such as the Decisional Bilinear Diffie–Hellman (DBDH), and their variants, in the sense that any successful adversary must implicitly solve such problems through generic operations.

Although our proofs are carried out in the generic group model rather than via explicit reductions to specific assumptions (e.g., DBDH), the security intuition is closely related to standard bilinear hardness assumptions. In particular, any adversary that succeeds in breaking the proposed schemes in the GGM would imply the ability to derive non-trivial relations among the underlying exponents, which is widely believed to be computationally infeasible. Therefore, our results provide strong evidence of security against generic adversaries, consistent with the security guarantees achieved by many existing pairing-based constructions.

Theorem 1: The proposed MA-KPABE scheme achieves adaptive IND-CPA security in the generic group model, assuming the hash function H is modeled as a random oracle.

Proof: In the IND-CPA security game, the only ciphertext component that depends on the challenged messages is $ct_4 = msg \cdot \prod_{k\in\Gamma} \hat{e}(g_1,g_2)^{\alpha_k s}$. Hence, the adversary $\mathcal{A}$ attempts to distinguish between $ct_4 = msg_0^* \cdot \prod_{k\in\Gamma} \hat{e}(g_1,g_2)^{\alpha_k s}$ and $ct_4 = msg_1^* \cdot \prod_{k\in\Gamma} \hat{e}(g_1,g_2)^{\alpha_k s}$.

For a random $\vartheta \xleftarrow{\$} \mathbb{Z}_p$ and $b \xleftarrow{\$} \{0,1\}$, the probability of distinguishing $msg_0^* \cdot \prod_{k\in\Gamma} \hat{e}(g_1,g_2)^{\alpha_k s}$ from $\hat{e}(g_1,g_2)^{\vartheta}$ is equivalent to distinguishing $\hat{e}(g_1,g_2)^{\vartheta}$ from $msg_1^* \cdot \prod_{k\in\Gamma} \hat{e}(g_1,g_2)^{\alpha_k s}$. Therefore, if $\mathcal{A}$ has advantage ε in the IND-CPA game, it achieves at most $\frac{\varepsilon}{2}$ advantage in distinguishing $\prod_{k\in\Gamma} \hat{e}(g_1,g_2)^{\alpha_k s}$ from $\hat{e}(g_1,g_2)^{\vartheta}$. We now define a modified game in which $\mathcal{A}$ attempts to distinguish these two values.

Setup: The $\mathcal{C}$ picks $MK_k = (\beta_k^1, \beta_k^2, \alpha_k) \xleftarrow{\$} \mathbb{Z}_p$ and publishes the public key $PK_k = (g_2^{\beta_k^1}, g_2^{\beta_k^2}, \hat{e}(g_1,g_2)^{\alpha_k})$ to $\mathcal{A}$.

Phase 1: $\mathcal{A}$ may query the random oracle and the key generation oracle. For an access structure $\mathbb{A} = (\mathbb{M}, \pi, \{\pi(i)\}_{i\in[\ell]})$, $\mathcal{C}$ picks $r_k \xleftarrow{\$} \mathbb{Z}_p$ and a vector $\vec{v}_k \xleftarrow{\$} \mathbb{Z}_p^{n-1}$, and defines $\lambda_{k,i} = \mathbb{M}_{u,i}^k(\alpha_k \| \vec{v}_k)^\top$. The corresponding secret key is generated as:

$$sk_1^k = g_2^{r_k}, sk_2^{k,i} = g_1^{(\lambda_{k,i}+t_i r_k)\cdot\frac{1}{\beta_k^1}}, sk_3^{k,i} = g_1^{(\lambda_{k,i}+t_i r_k)\cdot\frac{1}{\beta_k^2}}.$$

Finally, $\mathcal{C}$ returns $SK = (sk_1^k, \{sk_2^{k,i}, sk_3^{k,i}\}_{i\in[\ell],k\in\Gamma})$ to $\mathcal{A}$.

Challenge: $\mathcal{A}$ submits an attribute set $\mathbb{S}^*$ and two challenge messages msg_0^*, msg_1^*. $\mathcal{C}$ rejects $\mathbb{S}^*$ if it satisfies any previously queried access structure. Otherwise, it chooses $s_1, s_2, \vartheta \xleftarrow{\$} \mathbb{Z}_p$ and sets $s = s_1 + s_2$. If $\mu = 0$, the ciphertext is generated as:

$$ct_{2,k} = g_2^{\beta_k^1 s_1}, ct_{3,k} = g_2^{\beta_k^2 s_2}, ct_4 = \prod_{k\in\Gamma} \hat{e}(g_1,g_2)^{\alpha_k s}.$$

Otherwise, $\mathcal{C}$ sets:

$$ct_{1,i} = g_1^{t_i s},\ ct_{2,k} = g_2^{\beta_k^1 s_1},\ ct_{3,k} = g_2^{\beta_k^2 s_2},\ ct_4 = \hat{e}(g_1,g_2)^{\vartheta}.$$

The ciphertext CT is then returned to $\mathcal{A}$.

Phase 2: Same as Phase 1, with the restriction that no queried access structure satisfies $\mathbb{S}^*$.

We can see that if $\mathcal{A}$ can construct $\prod_{k\in\Gamma} \hat{e}(g_1,g_2)^{\delta_k\alpha_k s}$ for some $\delta_k \in \mathbb{Z}_p$ using its oracle queries. To do so, $\mathcal{A}$ may combine group elements as follows:

- $\mathbb{G}_1$: $1, t_i, t_i s, (\lambda_{k,i}+t_i r_k)/\beta_k^1, (\lambda_{k,i}+t_i r_k)/\beta_k^2$;
- $\mathbb{G}_2$: $1, r_k, \beta_k^1 s_1, \beta_k^2 s_2, \beta_k^1, \beta_k^2$;
- $\mathbb{G}_T$: α_k.

Since $s = s_1 + s_2$ cannot be derived through simple group operations, the only way to obtain a term involving s is by pairing $(\lambda_{k,i}+t_i r_k)/\beta_k^1$ with $\beta_k^1 s_1$ and $(\lambda_{k,i}+t_i r_k)/\beta_k^2$ with $\beta_k^2 s_2$, yielding $(\lambda_{k,i}+t_i r_k)(s_1+s_2) = \lambda_{k,i} s + t_i r_k s$ in $\mathbb{G}_T$.

To obtain $\delta_k\alpha_k s$, $\mathcal{A}$ must cancel the $\lambda_{k,i} s$ term and reconstruct $t_i r_k s$ using oracle outputs. However, as $\mathbb{S}^*$ does not satisfy any queried access structure, this reconstruction is infeasible. Thus, $\mathcal{A}$ cannot form $\delta_k\alpha_k s$ with non-negligible probability.

Consequently, $\mathcal{A}$ gains only negligible advantage in distinguishing the modified game, implying negligible advantage in the IND-CPA game. Hence, the Theorem 1 holds. □

Theorem 2: The proposed MA-KPABE scheme achieves anonymity under the generic group model, assuming the hash function H is modeled as a random oracle.

Proof: In the anonymity game, the only ciphertext component associated with the challenged attribute sets is $ct_{1,i} = H(x_i)^s$. This can be equivalently simulated as $ct_{1,i} = g_1^{t_i s}$. Hence, the $\mathcal{A}$ attempts to distinguish $\{g_1^{t_{i0} s}\}_{i\in[m]}$ from $\{g_1^{t_{i1} s}\}_{i\in[m]}$.

For $\vartheta \xleftarrow{\$} \mathbb{Z}_p$ and $b \xleftarrow{\$} \{0,1\}$, distinguishing $\{g_1^{t_{i0} s}\}_{i\in[m]}$ from $\{g_1^{t_{ib}\vartheta}\}_{i\in[m]}$ is equivalent to distinguishing $\{g_1^{t_{ib}\vartheta}\}_{i\in[m]}$ from $\{g_1^{t_{i1} s}\}_{i\in[m]}$. Therefore, if $\mathcal{A}$ has advantage ε in the anonymity game, it achieves at most $\frac{\varepsilon}{2}$ advantage in distinguishing $g_1^{t_{ib} s}$ from $g_1^{t_{ib}\vartheta}$. We thus consider a modified game where $\mathcal{A}$ aims to distinguish these two values.

Setup: The $\mathcal{C}$ selects $MK_k = (\beta_k^1, \beta_k^2, \alpha_k) \xleftarrow{\$} \mathbb{Z}_p$ and publishes the public key $PK_k = (g_2^{\beta_k^1}, g_2^{\beta_k^2}, e(g_1,g_2)^{\alpha_k})$ to $\mathcal{A}$.

Phase 1: This process is consistent with the execution procedure described in Theorem 1. The adversary $\mathcal{A}$ can query both the random oracle and the key generation oracle. For an access structure $\mathbb{A} = (\mathbb{M}, \pi, \{\pi(i)\}_{i\in[\ell]})$, $\mathcal{C}$ samples $r_k \xleftarrow{\$} \mathbb{Z}_p$ and a vector $\vec{v}_k \xleftarrow{\$} \mathbb{Z}_p^{n-1}$, and defines $\lambda_{k,i} = \mathbb{M}_{u,i}^k(\alpha_k \| \boldsymbol{v}_k)^\top$. Then, the secret key is computed as:

$$sk_1^k = g_2^{r_k}, sk_2^{k,i} = g_1^{(\lambda_{k,i}+t_i r_k)\cdot\frac{1}{\beta_k^1}}, sk_3^{k,i} = g_1^{(\lambda_{k,i}+t_i r_k)\cdot\frac{1}{\beta_k^2}}.$$

Finally, $\mathcal{C}$ returns $SK = (sk_1^k, \{sk_2^{k,i}, sk_3^{k,i}\}_{i\in[\ell],k\in\Gamma})$ to $\mathcal{A}$.

Challenge: $\mathcal{A}$ outputs two attribute sets $\mathbb{S}_0^* = \{x_{i0}\}_{i\in[m]}$ and $\mathbb{S}_1^* = \{x_{i1}\}_{i\in[m]}$ with identical attribute names $\{n_i\}_{i\in[m]}$. If either set satisfies any previously queried access structure, $\mathcal{C}$ aborts. Otherwise, $\mathcal{C}$ samples $s_1, s_2, \vartheta \xleftarrow{\$} \mathbb{Z}_p$ and lets $s = s_1 + s_2$. Then $\mathcal{C}$ randomly selects $b \xleftarrow{\$} \{0,1\}$ and $\mu \xleftarrow{\$} \{0,1\}$. If $\mu = 0$, $\mathcal{C}$ sets $ct_{1,i} = g_1^{t_i\vartheta}$; otherwise, $ct_{1,i} = g_1^{t_i s}$. The remaining ciphertext components are computed as:

$$ct_{2,k} = g_2^{\beta_k^1 s_1}, ct_{3,k} = g_2^{\beta_k^2 s_2}, ct_4 = \prod_{k\in\Gamma} \hat{e}(g_1,g_2)^{\alpha_k s}.$$

$\mathcal{C}$ returns $CT = (\{ct_{1,i}\}_{i\in[m]}, \{ct_{2,k}, ct_{3,k}\}_{k\in\Gamma}, ct_4)$ to $\mathcal{A}$.

Phase 2: This phase is identical to Phase 1, except that no queried access structure may satisfy $\mathbb{S}_0^*$ or $\mathbb{S}_1^*$.

We analyze whether $\mathcal{A}$ can construct $\prod_{k\in\Gamma} \hat{e}(g_1,g_2)^{\delta_k t_i s}$ for some $\delta_k \in \mathbb{Z}_p$ from oracle outputs. The relevant group elements include:

- $\mathbb{G}_1$: $1, t_i, t_i s, (\lambda_{k,i}+t_i r_k)/\beta_k^1, (\lambda_{k,i}+t_i r_k)/\beta_k^2$;
- $\mathbb{G}_2$: $1, r_k, \beta_k^1 s_1, \beta_k^2 s_2, \beta_k^1, \beta_k^2$;
- $\mathbb{G}_T$: $\alpha_k, \alpha_k s$.

Since $s = s_1 + s_2$ cannot be combined directly, $\mathcal{A}$ can only derive s-dependent terms by pairing $(\lambda_{k,i} + t_i r_k)/\beta_k^1$ with $\beta_k^1 s_1$ and $(\lambda_{k,i} + t_i r_k)/\beta_k^2$ with $\beta_k^2 s_2$, obtaining $(\lambda_{k,i} + t_i r_k)(s_1 + s_2) = \lambda_{k,i} s + t_i r_k s$ in $\mathbb{G}_T$.

To derive $\delta_k t_i s$, $\mathcal{A}$ would need to cancel the $\lambda_{k,i} s$ term and reconstruct $t_i r_k s$ from oracle queries. However, as none of the queried access structures satisfy the challenge attribute sets, this reconstruction is impossible. Thus, $\mathcal{A}$ cannot generate $\delta_k t_i s$ with non-negligible probability.

Consequently, $\mathcal{A}$ gains only negligible advantage in the modified game, implying a negligible advantage in the anonymity game. Hence, Theorem 2 holds. □

Theorem 3: The proposed MA-ASE scheme is adaptively IND-CKA secure under the generic group model, assuming that the hash function H behaves as a random oracle.

Proof: In the IND-CKA game, the only ciphertext component related to the two challenged keyword sets is $ct_{1,i} = H(x_i)^s$. Similarly, we can simulate $ct_{1,i} = H(x_i)^s = g_1^{t_i s}$. Therefore, the adversary $\mathcal{A}$ attempts to win the game by distinguishing $\{g_1^{t_{i0} s}\}_{i\in[m]}$ from $\{g_1^{t_{i1} s}\}_{i\in[m]}$.

For $\vartheta \xleftarrow{\$} \mathbb{Z}_p$ and $b \xleftarrow{\$} \{0,1\}$, the probability of distinguishing $\{g_1^{t_{i0} s}\}_{i\in[m]}$ from $g_1^{t_{ib}\vartheta}$ is equivalent to distinguishing $g_1^{t_{ib}\vartheta}$ from $\{g_1^{t_{i1} s}\}_{i\in[m]}$. Therefore, if $\mathcal{A}$ has advantage ε in winning the IND-CKA game, then it has advantage $\frac{\varepsilon}{2}$ in distinguishing $g_1^{t_{ib} s}$ from $g_1^{t_{ib}\vartheta}$. Thus, we consider a modified game where $\mathcal{A}$ can distinguish $g_1^{t_{ib} s}$ from $g_1^{t_{ib}\vartheta}$. For simplicity, we denote $g_1^{t_{ib} s}$ and $g_1^{t_{ib}\vartheta}$ as $g_1^{t_i s}$ and $g_1^{t_i \vartheta}$ respectively in the following discussion. The modified game is simulated as follows.

Setup: The $\mathcal{C}$ chooses $MK_k = (\beta_k^1, \beta_k^2, \alpha_k) \xleftarrow{\$} \mathbb{Z}_p$ and sends the public key $PK_k = (g_2^{\beta_k^1}, g_2^{\beta_k^2}, e(g_1, g_2)^{\alpha_k})$ to $\mathcal{A}$.

Phase 1: The adversary $\mathcal{A}$ may make oracle queries as follows. When $\mathcal{A}$ makes a trapdoor query for a keyword policy $\mathbb{P} = (\mathbb{M}, \pi, \{\pi(i)\}_{i\in[\ell]})$, $\mathcal{C}$ picks $r_k \xleftarrow{\$} \mathbb{Z}_p$ and a vector $\vec{v}_k \xleftarrow{\$} \mathbb{Z}_p^{n-1}$. Let $\lambda_{k,i} = \mathbb{M}_{k,i}(\alpha_k \| \vec{v}_k)^\top$. Then $\mathcal{C}$ generates the trapdoor $td_1^k = g_2^{r_k}$ and

$$td_2^{k,i} = g_1^{(\lambda_{k,i} + t_i r_k)\cdot \frac{1}{\beta_k^1}}, \quad td_3^{k,i} = g_1^{(\lambda_{k,i} + t_i r_k)\cdot \frac{1}{\beta_k^2}}.$$

Finally, $\mathcal{C}$ returns $TD = (td_1^k, \{td_2^{k,i}, td_3^{k,i}\}_{i\in[\ell],k\in\Gamma})$ to $\mathcal{A}$.

Challenge: The adversary $\mathcal{A}$ outputs two keyword sets $\mathbb{W}_0^* = \{x_{i0}\}_{i\in[m]} = \{n_i, v_{i0}\}_{i\in[m]}$ and $\mathbb{W}_1^* = \{x_{i1}\}_{i\in[m]} = \{n_i, v_{i1}\}_{i\in[m]}$, which must share the same keyword names $\{n_i\}_{i\in[m]}$. The challenger $\mathcal{C}$ checks if either $\mathbb{W}_0^*$ or $\mathbb{W}_1^*$ satisfies any keyword policy $\mathbb{P}$ queried in Phase 1. If so, $\mathcal{C}$ rejects the challenge. Otherwise, it chooses $s_1, s_2, \vartheta \xleftarrow{\$} \mathbb{Z}_p$ and lets $s = s_1 + s_2$. Then $\mathcal{C}$ selects $b \xleftarrow{\$} \{0,1\}$ for encrypting one of the two keyword sets, and chooses $\mu \in \{0,1\}$. If $\mu = 0$, it generates $ct_{1,i} = g_1^{t_i \vartheta}$; otherwise, it sets $ct_{1,i} = g_1^{t_i s}$. The remaining ciphertext components are computed as

$$ct_{2,k} = g_2^{\beta_k^1 s_1}, \quad ct_{3,k} = g_2^{\beta_k^2 s_2}, \quad ct_4 = \prod_{k\in\Gamma} e(g_1, g_2)^{\alpha_k s}.$$

Finally, $\mathcal{C}$ returns $CT = (\{ct_{1,i}\}_{i\in[m]}, \{ct_{2,k}, ct_{3,k}\}_{k\in\Gamma}, ct_4)$ to $\mathcal{A}$.

Phase 2: This phase is identical to Phase 1, except that any keyword policy $\mathbb{P}$ queried must not satisfy the challenge keyword sets $\mathbb{W}_0^*$ and $\mathbb{W}_1^*$.

This process is consistent with the execution procedure described in Theorem 1, ensuring the same simulation structure.

We observe that if $\mathcal{A}$ can construct $\prod_{k\in\Gamma} e(g_1, g_2)^{\delta_k t_i s}$ for some $\delta_k \in \mathbb{Z}_p$ from previously queried oracle outputs, then $\mathcal{A}$ can use it to distinguish $g_1^{t_i \vartheta}$ from $g_1^{t_i s}$. Hence, it suffices to prove that $\mathcal{A}$ can construct $\prod_{k\in\Gamma} e(g_1, g_2)^{\delta_k t_i s}$ only with negligible probability, implying that $\mathcal{A}$ gains no non-negligible advantage in the IND-CKA game.

We now analyze the exponent elements available to $\mathcal{A}$ in the groups $\mathbb{G}_1$, $\mathbb{G}_2$, and $\mathbb{G}_T$:

- $\mathbb{G}_1$: $1, t_i, t_i s, (\lambda_{k,i} + t_i r_k)/\beta_k^1, (\lambda_{k,i} + t_i r_k)/\beta_k^2$;
- $\mathbb{G}_2$: $1, r_k, \beta_k^1 s_1, \beta_k^2 s_2, \beta_k^1, \beta_k^2$;
- $\mathbb{G}_T$: $\alpha_k, \alpha_k s$.

Since $s = s_1 + s_2$ cannot be derived by any linear combination over these groups, $\mathcal{A}$ cannot obtain a term of the form $\delta_k t_i s$ without combining both partial exponents. Even if $\mathcal{A}$ attempts to form such a term through bilinear pairings, it cannot eliminate the dependency on $\lambda_{k,i}$, as reconstructing $\lambda_{k,i}$ to α_k requires satisfying the keyword policy, which is explicitly prohibited in the challenge. Therefore, constructing $\delta_k t_i s$ in $\mathbb{G}_T$ is infeasible.

Consequently, $\mathcal{A}$'s advantage in the modified game is negligible, which implies that it also has negligible advantage in the IND-CKA game. Hence, the Theorem 3 is proved. □

VII. Implementation and Performance

This section presents a comprehensive evaluation, and first outlines the functional comparison framework and experimental setup, including benchmark schemes, system parameters, and implementation environment. We then analyze the performance of the MA-KPABE and MA-ASE schemes in terms of storage overhead, computational cost, and scalability under various access structures. Finally, we compare it with state-of-the-art schemes, highlighting its efficiency and practicality in multi-authority encrypted search scenarios.

A. Function Comparison

To comprehensively evaluate the functionality of our proposed scheme, we compare it with a series of representative Attribute-Based Encryption (ABE) and Attribute-Based Searchable Encryption (ABSE) schemes, as summarized in Table I. The comparison focuses on six key aspects: 1) the underlying ABE type (KP-ABE or CP-ABE), 2) the support for fine-grained access control, 3) the presence of a multi-authority setting, 4) the ability to perform encrypted keyword search, 5) the capability of partial attribute hiding, and 6) the provision of anonymity.

In the early ABE schemes such as FAME [9] and FABEO [10], although both support expressive and fine-grained access control policies, they rely on a single trusted authority and do not enable keyword search over encrypted data. Subsequently, several single-authority ABSE schemes—such as CWD [7], FEASE [8], CXJ [5], and LPCR [12]—combine searchable encryption with

TABLE I
FUNCTIONALITY COMPARISONS OF EXISTING SCHEMES

Schemes	KP/CP-ABE	Fine-grained Access	Multi-authority	Encrypted Search	Partially Hidden	Anonymous
FAME [9]	KP/CP-ABE	✓	×	×	×	×
FABEO [10]	KP/CP-ABE	✓	×	×	×	×
CWD [7]	KP-ABE	✓	×	✓	×	×
FEASE [8]	KP-ABE	✓	×	✓	✓	✓
CXJ [5]	CP-ABE	✓	×	✓	✓	✓
LPCR [12]	CP-ABE	✓	×	✓	✓	✓
rABEKS [19]	CP-ABE	✓	×	✓	×	×
YZB [13]	CP-ABE	✓	×	✓	×	×
FAKS [11]	CP-ABE	✓	×	✓	×	×
RWS [14]	CP-ABE	✓	✓	×	×	×
WCD [15]	KP-ABE	✓	✓	×	×	×
XHM [16]	KP-ABE	✓	✓	×	×	×
ERS-ABE [17]	CP-ABE	✓	✓	✓	×	×
PGVMatch [18]	CP-ABE	✓	✓	✓	×	×
Ours	KP-ABE	✓	✓	✓	✓	✓

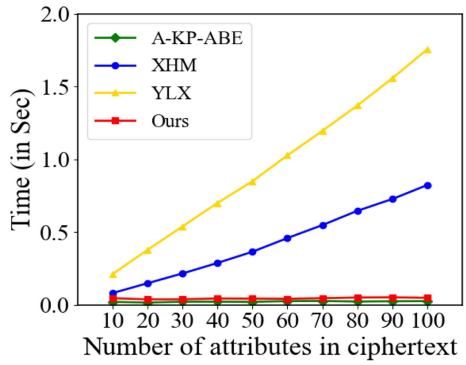


Fig. 6. Decryption with all-AND structure.

ABE to allow ciphertext retrieval under attribute-based access policies. However, these schemes are designed for centralized settings and lack multi-authority support, which limits their scalability and robustness in distributed environments.

To address single-point trust issues, multi-authority ABE schemes such as RWS [14], WCD [15], and XHM [16] have been proposed. These approaches effectively distribute the management of user attributes among multiple authorities, enhancing decentralization and reducing the risk of key escrow. Nevertheless, most existing multi-authority schemes do not incorporate searchable encryption capabilities, and therefore cannot support efficient query operations on encrypted datasets. More recently, ERS-ABE [17] and PGVMatch [18] extend multi-authority ABE to searchable settings, but they still expose partial attribute information, thereby compromising privacy.

In contrast, our proposed scheme is to achieve multi-authority key-policy ABE with encrypted search capability while simultaneously supporting partial attribute hiding and user anonymity. By introducing a decentralized authorization mechanism and a privacy-preserving trapdoor generation process, our design ensures that neither individual authorities nor the cloud server can infer sensitive attribute information during keyword search. This fills a gap in current research, where multi-authority KP-ABE schemes with searchable functionality and attribute privacy protection remain largely unexplored.

B. Experimental Setup

1) Symbol Definitions: To facilitate the reader's understanding, we first provide the description of parameters used in the performance evaluation. Specifically, $\mathsf{T}^{\mathsf{M}}_{\mathsf{G}_1}$, $\mathsf{T}^{\mathsf{M}}_{\mathsf{G}_2}$, and $\mathsf{T}^{\mathsf{M}}_{\mathsf{G}_\mathsf{T}}$ denote the multiplication time in groups $\mathbb{G}_1$, $\mathbb{G}_2$, and $\mathbb{G}_T$, respectively, whereas $\mathsf{T}^{\mathsf{E}}_{\mathsf{G}_1}$, $\mathsf{T}^{\mathsf{E}}_{\mathsf{G}_2}$, and $\mathsf{T}^{\mathsf{E}}_{\mathsf{G}_\mathsf{T}}$ represent the exponentiation time in these groups. The hash time into $\mathbb{G}_1$ and $\mathbb{G}_2$ is denoted by $\mathsf{T}^{\mathsf{H}}_{\mathsf{G}_1}$ and $\mathsf{T}^{\mathsf{H}}_{\mathsf{G}_2}$, respectively, while $\mathsf{T}_{\mathsf{Pair}}$ denotes the time for a bilinear pairing operation. We denote k as the number of attribute authorities, m as the size of the attribute/keyword set, a as the number of attributes/keywords managed by each authority, and ℓ as the size of an access structure or keyword policy. Moreover, x_1 denotes the total number of matched attribute/keyword name subsets, x_2 the number of attributes/keywords within each matched subset, and x_3 the number of matched subsets managed by a single authority. Finally, $|\mathbb{G}_1|$, $|\mathbb{G}_2|$, $|\mathbb{G}_T|$, and $|\mathbb{Z}_p|$ denote the size of an element in $\mathbb{G}_1$, $\mathbb{G}_2$, $\mathbb{G}_T$, and $\mathbb{Z}_p$, respectively.

2) Experimental Environment: All simulation experiments are conducted on Ubuntu 18.04 running within VMware 17.5.2 on a laptop equipped with Windows 11, an Intel(R) Core(TM) Ultra 5 125H processor at 3.60 GHz, and 16 GB RAM. The implementation is developed in Python 3.6 using the Charm 0.5 framework [34], with the MNT224 curve adopted for pairings, which serves as the default pairing library in Charm.

3) Implementation Details: For both access structures and keyword policies, Boolean formulas are first specified and then converted into Monotone Span Programs (MSPs) via the Lewko–Waters method [35]. Under this transformation, the resulting matrix $\mathbb{M}$ contains entries restricted to $\{0, 1, -1\}$, and the reconstruction coefficients $\{y^k_{u,i}\}$ are limited to $\{0, 1\}$, which considerably reduces computational overhead.

To provide a detailed performance evaluation on the MNT224 curve, we measured the execution times of multiplication, exponentiation, hashing, and pairing operations in groups $\mathbb{G}_1$, $\mathbb{G}_2$, and $\mathbb{G}_T$. The results, reported in milliseconds, are summarized in Table II.

It is worth noting that a partially hidden structure is employed to preserve attribute and keyword privacy, with only attribute or keyword names revealed. Consequently, during decryption, multiple candidate subsets of matching attribute names may

TABLE II
NOTATIONS OF EXECUTION TIME

Notations	Descriptions	Times
$T^{M}_{G_1}$	Multiplication time in group $\mathbb{G}_1$	0.01ms
$T^{M}_{G_2}$	Multiplication time in group $\mathbb{G}_2$	0.04ms
$T^{M}_{G_T}$	Multiplication time in group $\mathbb{G}_T$	0.02ms
$T^{E}_{G_1}$	Exponentiation time in group $\mathbb{G}_1$	1.67ms
$T^{E}_{G_2}$	Exponentiation time in group $\mathbb{G}_2$	10.4ms
$T^{E}_{G_T}$	Exponentiation time in group $\mathbb{G}_T$	1.94ms
$T^{H}_{G_1}$	Hash time in group $\mathbb{G}_1$	0.13ms
$T^{H}_{G_2}$	Hash time in group $\mathbb{G}_2$	28.15ms
T_{Pair}	Time for a bilinear pairing	7.58ms

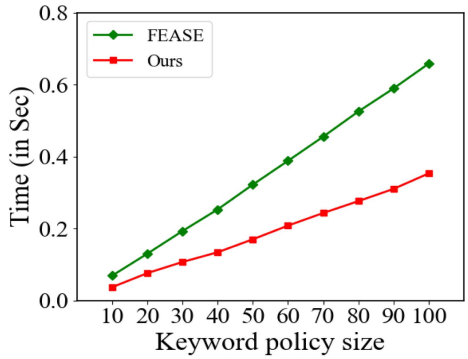


Fig. 7. Running times for Trapdoor Generation.

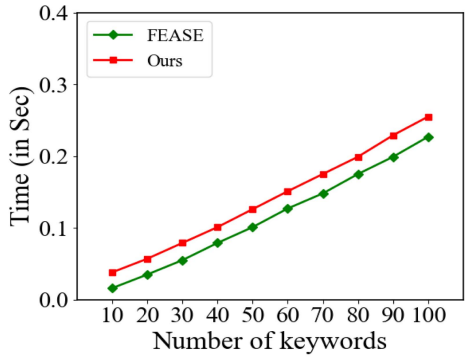


Fig. 8. Running times for Encryption.

be generated. Taking MA-KPABE as an example, in the decryption phase, even if the attribute names satisfy the access structure and the generated coefficients $\{y^k_{u,i} \in \mathbb{Z}_p\}_{i\in\mathbb{I}_k}$ fulfill $\sum_{i\in\mathbb{I}_k} y^k_{u,i} \cdot \mathbb{M}^k_{u,i} = (1, 0, \ldots, 0)$, decryption will still fail if the attribute values are incorrect. In such cases, the system must proceed to evaluate the next feasible subset.

C. Analysis on the Performance of MA-KPABE

For a comprehensive comparative evaluation, we consider three representative ABE schemes, namely A-KP-ABE [8], XHM [16], and YLX [17], in addition to our proposed MA-KPABE scheme. Notably, the work in [8] also introduces a public-key searchable encryption scheme, FEASE, which is used in subsequent comparisons.

A-KP-ABE is a recently proposed fast single-authority KP-ABE construction, which we implemented for comparison. XHM represents a multi-authority KP-ABE scheme, while YLX is a recently developed multi-authority CP-ABE scheme. The functional characteristics of these schemes are summarized in Table I, covering their ABE types, multi-authority support, anonymity properties, as well as their efficiency.

In both A-KP-ABE and our MA-KPABE scheme, attribute names are randomly selected from the English vocabulary, and each attribute is assigned a random positive integer between 1 and 1000 as its corresponding value. As a result, attributes take the form of key–value pairs such as "job: 223", "age: 42", and "address: 741". The integer values are fed into the encryption algorithm, whereas the attribute names remain publicly visible. By contrast, in XHM and YLX, attribute privacy protection is not a design objective, and thus partially hidden structures are unnecessary. Attributes in these schemes are exposed directly as plain names such as "job", "age", or "address".

1) Evaluation of Storage Cost: The comparison of storage overhead is summarized in the upper part of Table III, where we analyze the requirements from three perspectives: master secret key, secret key, and ciphertext.

For the master secret key maintained by each attribute authority, both our proposed MA-KPABE and the single-authority A-KP-ABE scheme incur a constant storage cost of $3|\mathbb{Z}_p|$, which is independent of the number of managed attributes. In contrast, the storage cost of XHM and YLX grows linearly with the number of attributes a, requiring $(2a + 2)|\mathbb{Z}_p|$ and $(3a + 1)|\mathbb{Z}_p|$, respectively. This linear growth makes XHM and YLX significantly less efficient when the authority manages a large attribute universe.

For the secret key, XHM incurs a storage cost of $2\ell|\mathbb{G}_1|$, where ℓ represents the size of the access structure. Compared to this, A-KP-ABE adds one extra element in $\mathbb{G}_2$, while our MA-KPABE introduces k additional elements in $\mathbb{G}_2$ due to the multi-authority setting. Although the inclusion of k extra elements introduces a slightly higher cost, this is the price paid to achieve stronger security guarantees and support for multi-authority environments.

Regarding the ciphertext, our MA-KPABE requires $m|\mathbb{G}_1| + 2k|\mathbb{G}_2| + |\mathbb{G}_T|$, which is marginally larger than A-KP-ABE. By comparison, XHM incurs a ciphertext size of $m|\mathbb{G}_2|$, while YLX requires $3\ell|\mathbb{G}_1|$. Both XHM and YLX result in ciphertext sizes that are generally larger than those of our scheme, especially in large-scale attribute or policy settings.

Overall, MA-KPABE achieves a compact master secret key and incurs only modest extra costs in the secret key and ciphertext. Compared with XHM and YLX, whose storage grows linearly with attributes, our scheme remains far more efficient while providing stronger security and functionality in multi-authority settings.

2) Evaluation of Computation Cost: The execution times of the ABE schemes are illustrated in Figs. 2–6. In all experiments, the number of attribute authorities was fixed, except during the AASetup phase. These observations are consistent with our theoretical analysis, summarized in the upper portions of Tables IV and V. The tables specify, for each scheme, the counts

TABLE III
THE COMPARISONS OF STORAGE COST BETWEEN EXISTING SCHEMES AND OURS

Schemes	Master Secret Key	Secret Key/Trapdoor	Ciphertext
A-KP-ABE [8]	$3\|\mathbb{Z}_p\|$	$2\ell\|\mathbb{G}_1\| + \|\mathbb{G}_2\|$	$m\|\mathbb{G}_1\| + 2\|\mathbb{G}_2\| + \|\mathbb{G}_T\|$
XHM [16]	$(2a+2)\|\mathbb{Z}_p\|$	$2\ell\|\mathbb{G}_1\|$	$m\|\mathbb{G}_2\| + \|\mathbb{G}_T\|$
YLX [17]	$(3a+1)\|\mathbb{Z}_p\|$	$(2m+2)\|\mathbb{G}_2\|$	$\|\mathbb{Z}_p\| + 3\ell\|\mathbb{G}_1\| + \|\mathbb{G}_T\|$
MA-KPABE	$3\|\mathbb{Z}_p\|$	$2\ell\|\mathbb{G}_1\| + k\|\mathbb{G}_2\|$	$m\|\mathbb{G}_1\| + 2k\|\mathbb{G}_2\| + \|\mathbb{G}_T\|$
FEASE [8]	$3\|\mathbb{Z}_p\|$	$2\ell\|\mathbb{G}_1\| + \|\mathbb{G}_2\|$	$m\|\mathbb{G}_1\| + 2\|\mathbb{G}_2\| + \|\mathbb{G}_T\|$
MA-ASE	$3\|\mathbb{Z}_p\|$	$2\ell\|\mathbb{G}_1\| + k\|\mathbb{G}_2\|$	$m\|\mathbb{G}_1\| + 2k\|\mathbb{G}_2\| + \|\mathbb{G}_T\|$

TABLE IV
COMPUTATIONAL COST FOR AASETUP AND KEY/TRAPDOOR GENERATION

Schemes	AASetup	Secret Key/Trapdoor Generation
A-KP-ABE [8]	$2 \cdot \mathsf{T}^{\mathsf{E}}_{\mathsf{G}_2} + \mathsf{T}^{\mathsf{E}}_{\mathsf{G}_\mathsf{T}} + \mathsf{T}_{\mathsf{Pair}}$	$\ell \cdot \mathsf{T}^{\mathsf{M}}_{\mathsf{G}_1} + 4\ell \cdot \mathsf{T}^{\mathsf{E}}_{\mathsf{G}_1} + \ell \cdot \mathsf{T}^{\mathsf{H}}_{\mathsf{G}_1} + \mathsf{T}^{\mathsf{E}}_{\mathsf{G}_2}$
XHM [16]	$m \cdot \mathsf{T}^{\mathsf{E}}_{\mathsf{G}_2} + k \cdot \mathsf{T}^{\mathsf{E}}_{\mathsf{G}_\mathsf{T}} + \mathsf{T}_{\mathsf{Pair}}$	$3\ell \cdot \mathsf{T}^{\mathsf{E}}_{\mathsf{G}_1} + m \cdot \mathsf{T}^{\mathsf{E}}_{\mathsf{G}_2}$
YLX [17]	$(2m+n) \cdot \mathsf{T}^{\mathsf{E}}_{\mathsf{G}_1}$	$m \cdot \mathsf{T}^{\mathsf{M}}_{\mathsf{G}_2} + (3m+2) \cdot \mathsf{T}^{\mathsf{E}}_{\mathsf{G}_2} + \mathsf{T}^{\mathsf{H}}_{\mathsf{G}_2}$
MA-KPABE	$2k \cdot \mathsf{T}^{\mathsf{E}}_{\mathsf{G}_2} + k \cdot \mathsf{T}^{\mathsf{E}}_{\mathsf{G}_\mathsf{T}} + \mathsf{T}_{\mathsf{Pair}}$	$\ell \cdot \mathsf{T}^{\mathsf{M}}_{\mathsf{G}_1} + 4\ell \cdot \mathsf{T}^{\mathsf{E}}_{\mathsf{G}_1} + \ell \cdot \mathsf{T}^{\mathsf{H}}_{\mathsf{G}_1} + k \cdot \mathsf{T}^{\mathsf{E}}_{\mathsf{G}_2}$
FEASE [8]	$2 \cdot \mathsf{T}^{\mathsf{E}}_{\mathsf{G}_2} + \mathsf{T}^{\mathsf{E}}_{\mathsf{G}_\mathsf{T}} + \mathsf{T}_{\mathsf{Pair}}$	$2\ell \cdot \mathsf{T}^{\mathsf{M}}_{\mathsf{G}_1} + 4\ell \cdot \mathsf{T}^{\mathsf{E}}_{\mathsf{G}_1} + \ell \cdot \mathsf{T}^{\mathsf{H}}_{\mathsf{G}_1} + \mathsf{T}^{\mathsf{E}}_{\mathsf{G}_2}$
MA-ASE	$2k \cdot \mathsf{T}^{\mathsf{E}}_{\mathsf{G}_2} + k \cdot \mathsf{T}^{\mathsf{E}}_{\mathsf{G}_\mathsf{T}} + \mathsf{T}_{\mathsf{Pair}}$	$\ell \cdot \mathsf{T}^{\mathsf{M}}_{\mathsf{G}_1} + 4\ell \cdot \mathsf{T}^{\mathsf{E}}_{\mathsf{G}_1} + \ell \cdot \mathsf{T}^{\mathsf{H}}_{\mathsf{G}_1} + k \cdot \mathsf{T}^{\mathsf{E}}_{\mathsf{G}_2}$

TABLE V
COMPUTATIONAL COST FOR ENCRYPTION AND DECRYPTION/SEARCH

Schemes	Encryption	Decryption/Search
A-KP-ABE [8]	$m \cdot (\mathsf{T}^{\mathsf{E}}_{\mathsf{G}_1} + \mathsf{T}^{\mathsf{H}}_{\mathsf{G}_1}) + 2 \cdot \mathsf{T}^{\mathsf{E}}_{\mathsf{G}_2} + \mathsf{T}^{\mathsf{M}}_{\mathsf{G}_\mathsf{T}} + \mathsf{T}^{\mathsf{E}}_{\mathsf{G}_\mathsf{T}}$	$3x_2 \cdot \mathsf{T}^{\mathsf{M}}_{\mathsf{G}_1} + 3 \cdot \mathsf{T}^{\mathsf{M}}_{\mathsf{G}_\mathsf{T}} + 3x_1 \cdot \mathsf{T}_{\mathsf{Pair}}$
XHM [16]	$m \cdot \mathsf{T}^{\mathsf{E}}_{\mathsf{G}_2} + k \cdot (\mathsf{T}^{\mathsf{M}}_{\mathsf{G}_\mathsf{T}} + \mathsf{T}^{\mathsf{E}}_{\mathsf{G}_\mathsf{T}})$	$x_2 \cdot (\mathsf{T}^{\mathsf{M}}_{\mathsf{G}_1} + \mathsf{T}^{\mathsf{E}}_{\mathsf{G}_1} + \mathsf{T}^{\mathsf{M}}_{\mathsf{G}_\mathsf{T}} + \mathsf{T}_{\mathsf{Pair}})$
YLX [17]	$2\ell \cdot \mathsf{T}^{\mathsf{M}}_{\mathsf{G}_1} + 5\ell \cdot \mathsf{T}^{\mathsf{E}}_{\mathsf{G}_1} + \mathsf{T}^{\mathsf{M}}_{\mathsf{G}_\mathsf{T}} + \mathsf{T}^{\mathsf{E}}_{\mathsf{G}_\mathsf{T}}$	$3x_2 \cdot (\mathsf{T}^{\mathsf{M}}_{\mathsf{G}_\mathsf{T}} + \mathsf{T}_{\mathsf{Pair}})$
MA-KPABE	$m \cdot (\mathsf{T}^{\mathsf{E}}_{\mathsf{G}_1} + \mathsf{T}^{\mathsf{H}}_{\mathsf{G}_1}) + k \cdot (2 \cdot \mathsf{T}^{\mathsf{E}}_{\mathsf{G}_2} + \mathsf{T}^{\mathsf{M}}_{\mathsf{G}_\mathsf{T}} + \mathsf{T}^{\mathsf{E}}_{\mathsf{G}_\mathsf{T}})$	$3x_2 \cdot \mathsf{T}^{\mathsf{M}}_{\mathsf{G}_1} + 3x_2^n \cdot \mathsf{T}^{\mathsf{M}}_{\mathsf{G}_\mathsf{T}} + 3x_1 \cdot \mathsf{T}_{\mathsf{Pair}}$
FEASE [8]	$m \cdot (\mathsf{T}^{\mathsf{E}}_{\mathsf{G}_1} + \mathsf{T}^{\mathsf{H}}_{\mathsf{G}_1}) + 2 \cdot \mathsf{T}^{\mathsf{E}}_{\mathsf{G}_2} + \mathsf{T}^{\mathsf{E}}_{\mathsf{G}_\mathsf{T}}$	$3x_2 \cdot \mathsf{T}^{\mathsf{M}}_{\mathsf{G}_1} + 2 \cdot \mathsf{T}^{\mathsf{M}}_{\mathsf{G}_2} + 3x_1 \cdot \mathsf{T}_{\mathsf{Pair}}$
MA-ASE	$m \cdot (\mathsf{T}^{\mathsf{E}}_{\mathsf{G}_1} + \mathsf{T}^{\mathsf{H}}_{\mathsf{G}_1}) + k \cdot (2 \cdot \mathsf{T}^{\mathsf{E}}_{\mathsf{G}_2} + \mathsf{T}^{\mathsf{M}}_{\mathsf{G}_\mathsf{T}} + \mathsf{T}^{\mathsf{E}}_{\mathsf{G}_\mathsf{T}})$	$3x_2 \cdot \mathsf{T}^{\mathsf{M}}_{\mathsf{G}_1} + 2x_2^n \cdot \mathsf{T}^{\mathsf{M}}_{\mathsf{G}_2} + 3x_1 \cdot \mathsf{T}_{\mathsf{Pair}}$

of multiplications, exponentiations, and hash operations within each group, as well as the number of required pairing operations.

Computation Cost of AASetup Algorithm: We evaluate the time required to generate the master secret key and public key under scenarios involving 2 to 9 attribute authorities. Since the A-KP-ABE scheme [8] is a centralized single-authority KP-ABE construction, it does not support a multi-authority setting. Consequently, its performance remains independent of the number of attribute authorities and is therefore excluded from this comparison. Instead, we benchmark our scheme against XHM [16] and YLX [17].

As shown in Fig. 2, our proposed MA-KPABE scheme requires only 0.28 s to generate keys with 9 attribute authorities, which is 0.06 s faster than YLX. In contrast, XHM incurs 1.15 s in the same setting, making it nearly four times slower than our scheme. This performance advantage stems from the fact that both XHM and YLX require each attribute authority to predefine the attributes under its management and generate the corresponding cryptographic parameters for each attribute. In contrast, our MA-KPABE eliminates this requirement, resulting in significantly reduced computational overhead. As summarized in Table IV, XHM requires m exponentiations in $\mathbb{G}_2$, whereas YLX involves $2m+n$ exponentiations in $\mathbb{G}_1$.

Computation Cost of KeyGen Algorithm: As shown in Fig. 3, the proposed MA-KPABE scheme achieves the lowest execution time in the KeyGen phase. For an access structure with 100 attributes, our scheme performs this stage in only 0.32 s, which is nearly twice as fast as A-KP-ABE, XHM, and YLX. This performance advantage primarily stems from the elimination of redundant computations in our design. As summarized in Table IV, our scheme requires k exponentiations in $\mathbb{G}_2$, the computation time per authority in our multi-authority setting remains significantly shorter. The key reason is that our scheme delegates operations to each authority, thereby avoiding the repetitive attribute parameter generation required in A-KP-ABE and XHM.

In addition, the efficiency gap becomes more evident as the access structure grows. In A-KP-ABE and XHM, the computation cost increases rapidly with the number of attributes, leading to higher latency. By contrast, YLX, as a CP-ABE scheme, incurs extra related costs, which further amplify the performance disparity. Overall, these results confirm that our MA-KP-ABE not only supports multiple authorities but also offers superior scalability in managing large access structures.

Computation Cost of Encryption Algorithm: We evaluated the encryption performance of the four schemes using attribute sets of sizes $10, 20, \ldots, 100$. As shown in Fig. 4, our MA-KPABE scheme consistently demonstrates the best performance among the multi-authority constructions. For an access structure with 100 attributes, MA-KPABE completes encryption in 0.21 s. In contrast, XHM and YLX require 1.00 s and 1.47 s, respectively, making them approximately five and seven times slower than our scheme. Moreover, the performance curves indicate that the running time of MA-KP-ABE grows almost linearly

with the number of attributes, but the slope is significantly lower than that of XHM and YLX, highlighting its superior scalability.

As summarized in Table V, the single-authority A-KP-ABE scheme requires only 2 exponentiations in $\mathbb{G}_2$ and 1 exponentiation in $\mathbb{G}_T$, whereas our MA-KPABE involves k times these operations due to the participation of k attribute authorities. To ensure a fair comparison, we set $k = 1$ in our evaluation, aligning our scheme with the single-authority design of A-KP-ABE. Under this configuration, MA-KPABE achieves nearly identical performance to A-KP-ABE, confirming that the additional flexibility of supporting multiple authorities does not impose significant overhead in the single-authority case.

Computation Cost of Decryption Algorithm: To simplify the experiments, we set the number of attributes m equal to the size of the access structure l, varying the number of attributes from 10 to 100. Fig. 5 presents the decryption performance under access structures composed solely of "OR" gates. XHM and YLX exhibit nearly constant decryption times of 0.02 s and 0.03 s, respectively, while both MA-KPABE and A-KP-ABE show linear growth as the number of matched attributes increases. Although ring-based access structures are rarely used in practice, our MA-KPABE is only 0.4 s slower than XHM and YLX when $x_3 = 10$. As shown in Fig. 6, under "AND" gate access structures, MA-KPABE decrypts 100 attributes in 0.05 s and remains nearly stable, increasing by only 0.02 s compared to A-KP-ABE. Under the same conditions, XHM and YLX demonstrate linear increases in decryption time, requiring 0.82 s and 1.76 s, respectively—both significantly slower than MA-KPABE. As summarized in Table V, when $x_1 = 1$, both MA-KPABE and A-KP-ABE require a constant number of 3 pairings, which is considerably fewer than the x_2 pairings in XHM and $3x_2$ pairings in YLX. Although MA-KPABE introduces additional exponentiations in $\mathbb{G}_2$, making it slightly slower than A-KP-ABE, it achieves higher security and greater flexibility by supporting decentralized multi-authority settings. Therefore, this minor performance trade-off is acceptable. Moreover, MA-KPABE incorporates partially hidden structures to enhance attribute privacy protection.

D. Analysis on the Performance of MA-ASE

For the ASE scheme, we chose to compare our MA-ASE with the FEASE scheme from the single-authority ASE scheme [8]. The format of keywords and keyword policies follows the same approach used in implementing MA-KPABE, such as "prospect: 5", "initiative: 8", and "paradigm: 6".

1) Evaluation of Storage Cost: The storage overhead comparison between FEASE and MA-ASE is summarized in Table III. For the master secret key, both schemes require only three elements in $\mathbb{Z}_p$, resulting in identical storage costs for the authority. Regarding the trapdoor, FEASE requires $2\ell|\mathbb{G}_1| + |\mathbb{G}_2|$, while MA-ASE requires $2\ell|\mathbb{G}_1| + k|\mathbb{G}_2|$. This implies that MA-ASE introduces $(k-1)$ additional elements in $\mathbb{G}_2$ compared with FEASE. For the ciphertext, FEASE contains $m|\mathbb{G}_1| + 2|\mathbb{G}_2| + |\mathbb{G}_T|$, whereas MA-ASE contains $m|\mathbb{G}_1| + 2k|\mathbb{G}_2| + |\mathbb{G}_T|$, thereby introducing $2(k-1)$ additional elements in $\mathbb{G}_2$.

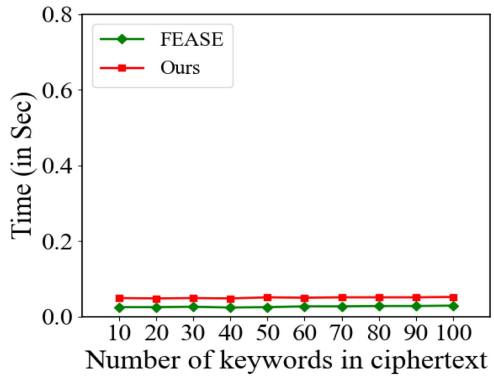


Fig. 9. Running times for Searchable.

Overall, the difference in storage cost between MA-ASE and FEASE lies solely in the number of $\mathbb{G}_2$ elements, while the $\mathbb{Z}_p$, $\mathbb{G}_1$, and $\mathbb{G}_T$ components remain identical. In practice, the number of authorities k is typically small (e.g., 2–5), leading to only a minor increase in storage overhead. For instance, assuming one element in $\mathbb{G}_2$ occupies 48 bytes, when $k = 3$, the additional storage introduced by MA-ASE is less than 300 bytes in total, which is negligible in modern systems. Therefore, the additional storage cost incurred by MA-ASE is acceptable and provides a reasonable trade-off for supporting a decentralized multi-authority structure with enhanced flexibility and security.

2) Evaluation of Computation Cost: The execution times of the ASE schemes are depicted in Figs. 7–9. Consistent with the experiments conducted for the aforementioned ABE schemes, the number of authorities was fixed at 2 in all tests. These results align with our theoretical summary provided in the lower sections of Tables IV and V.

Computation Cost of AASetup Algorithm: We evaluate the computation cost of the AASetup algorithm for the FEASE and MA-ASE schemes. As summarized in Table IV, the FEASE scheme requires $2\mathsf{T}^{\mathsf{E}}_{\mathsf{G}_2} + \mathsf{T}^{\mathsf{E}}_{\mathsf{G}_\mathsf{T}} + \mathsf{T}_{\mathsf{Pair}}$ operations. For our MA-ASE scheme, the computation cost increases to $2k\mathsf{T}^{\mathsf{E}}_{\mathsf{G}_2} + k\mathsf{T}^{\mathsf{E}}_{\mathsf{G}_\mathsf{T}} + \mathsf{T}_{\mathsf{Pair}}$, where k is the number of attribute authorities. This linear increase results from the decentralized design, as each authority independently generates its own public and secret parameters to ensure autonomous key management. The experimental evaluation results are similar to those shown in Fig. 2, where the MA-ASE scheme required only 0.28 s to complete the AASetup algorithm under a configuration of 9 attribute authorities, demonstrating high computational efficiency.

Computation Cost of TrapGen Algorithm: We evaluate the computation cost of the TrapGen algorithm by selecting keyword sets of sizes $10, 20, \ldots, 100$ and constructing access policies using both "AND" and "OR" gates among these keywords. Similar to the previously described multi-authority KP-ABE experiments, the computation time is measured under a single-authority configuration for consistency. As shown in Fig. 7, the proposed MA-ASE scheme requires only 0.35 s to generate a trapdoor for 100 keywords, while FEASE incurs nearly twice this computation time. The main reason is that, in FEASE, all trapdoors are generated by a centralized authority, whereas in our scheme, the computation workload is distributed

among multiple authorities. Consequently, the experimental results demonstrate that MA-ASE achieves lower per-authority computation overhead and higher scalability in multi-authority settings.

Computation Cost of Encrypt Algorithm: We evaluate the computation cost of the Encryption algorithm using keyword sets of sizes $10, 20, \ldots, 100$. As illustrated in Fig. 8, both MA-ASE and FEASE exhibit comparable encryption times, ranging from 0.23 s to 0.25 s when encrypting 100 keywords. Although MA-ASE introduces an additional lightweight operation, denoted as $\mathsf{T}^{\mathsf{M}}_{\mathsf{G}_\mathsf{T}}$, during the encryption phase, this extra cost does not significantly impact the overall performance. The primary reason lies in the difference in computational distribution between the two schemes. In FEASE, all encryption tasks are executed by a centralized authority, resulting in concentrated computational overhead. In contrast, MA-ASE distributes the encryption workload among multiple authorities, allowing the operations to be executed in parallel. Experimental results confirm that, despite the inclusion of $\mathsf{T}^{\mathsf{M}}_{\mathsf{G}_\mathsf{T}}$, MA-ASE achieves nearly identical encryption times while demonstrating better scalability and efficiency in multi-authority environments.

Computation Cost of Search Algorithm: We evaluate the computation cost of the Searchable algorithm by setting the number of keywords m equal to the size of the keyword policy l and varying the keyword quantity from 10 to 100 under a strict "AND" policy. As illustrated in Fig. 9, the proposed MA-ASE scheme completes the search for 100 keywords in only 0.05 s, exhibiting nearly constant performance with merely a 0.02 s delay compared with FEASE. As summarized in Table V, when $x_1 = 1$, both MA-ASE and FEASE require three pairing operations in $\mathbb{G}_T$. MA-ASE performs additional exponentiations in $\mathbb{G}_2$, resulting in slightly higher computation time. Nevertheless, since MA-ASE distributes the computation across multiple authorities, it achieves improved scalability and security in decentralized settings, making this minor performance difference acceptable.

VIII. Conclusion

This paper proposes a policy-hiding multi-authority key-policy attribute-based Encryption scheme with expressive keyword search (MA-ASE) to overcome the limitations of centralized trust and information leakage in existing systems. Built upon a novel policy-hiding MA-KPABE foundation, our construction enables expressive keyword search while concealing sensitive access structures and user attributes under a decentralized trust model. Formal security proofs demonstrate its resilience against chosen-plaintext, chosen-keyword, and chosen-access-policy attacks. Performance evaluations confirm that the distributed architecture significantly reduces authority-side overhead, providing a practical and scalable solution for secure, fine-grained data retrieval in cloud environments.

Acknowledgment

The authors thank the editor and anonymous reviewers for their valuable comments.

References

[1] X. Wang et al., "Attribute-based access control encryption," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 3, pp. 2227–2242, May/Jun. 2025.
[2] J. Tian, Y. Lu, and J. Li, "Lightweight searchable and equality-testable certificateless authenticated encryption for encrypted cloud data," *IEEE Trans. Mobile Comput.*, vol. 23, no. 8, pp. 8431–8446, Aug. 2024.
[3] C. Zhao, L. Xu, J. Li, H. Fang, and Y. Zhang, "Toward secure and privacy-preserving cloud data sharing: Online/offline multiauthority CP-ABE with hidden policy," *IEEE Syst. J.*, vol. 16, no. 3, pp. 4804–4815, Sep. 2022.
[4] B. Zhang, W. Yang, F. Zhang, and J. Ning, "Efficient attribute-based searchable encryption with policy hiding over personal health records," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 2, pp. 1299–1312, Mar./Apr. 2025.
[5] L. Chen, S. Xu, C. Jin, H. Zhang, and J. Weng, "Partially hidden policy attribute-based multi-keyword searchable encryption with verification and revocation," *IEEE Trans. Mobile Comput.*, vol. 24, no. 9, pp. 9020–9035, Sep. 2025.
[6] R. Hou, F. Zhou, Q. Wang, Z. Jiao, J. Sun, and Z. Zhang, "Revokable blockchain-enabled ranked multi-keyword attribute-based searchable encryption scheme with mobile edge computing for vehicular," *IEEE Trans. Netw. Service Manag.*, vol. 22, no. 3, pp. 2764–2779, Jun. 2025.
[7] H. Cui, Z. Wan, R. H. Deng, G. Wang, and Y. Li, "Efficient and expressive keyword search over encrypted data in cloud," *IEEE Trans. Dependable Secur. Comput.*, vol. 15, no. 3, pp. 409–422, May/Jun. 2018.
[8] L. Meng, L. Chen, Y. Tian, M. Manulis, and S. Liu, "FEASE: Fast and expressive asymmetric searchable encryption," in *Proc. USENIX Secur. Symp.*, 2024, pp. 2545–2562.
[9] S. Agrawal and M. Chase, "Fame: Fast attribute-based message encryption," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 665–682.
[10] D. Riepel and H. Wee, "FABEO: Fast attribute-based encryption with optimal security," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2022, pp. 2491–2504.
[11] Q. Huang, Q. Wei, G. Yan, L. Zou, and Y. Yang, "Fast and privacy-preserving attribute-based keyword search in cloud document services," *IEEE Trans. Services Comput.*, vol. 16, no. 5, pp. 3348–3360, Sep./Oct. 2023.
[12] N. Wang, W. Zhou, Q. Han, J. Liu, W. Liao, and J. Fu, "A lightweight privacy-preserving ciphertext retrieval scheme based on edge computing," *IEEE Trans. Cloud Comput.*, vol. 12, no. 4, pp. 1273–1290, Fourth Quarter 2024.
[13] Z. Yan and B. Zhang, "An efficient attribute-based multikeyword searchable encryption with access policy hiding in IoT using blockchain," *IEEE Internet Things J.*, vol. 12, no. 15, pp. 32148–32160, Aug. 2025.
[14] Y. Rouselakis and B. Waters, "Efficient statically-secure large-universe multi-authority attribute-based encryption," in *Proc. Int. Conf. Financial Cryptogr. Data Secur.*, 2015, pp. 315–332.
[15] M. Wang, Z. Cao, X. Dong, R. Du, and J. Chen, "Decentralized multi-authority KP-ABE scheme without bilinear pairings," *IEEE Internet Things J.*, vol. 12, no. 1, pp. 726–738, Jan. 2025.
[16] M. Xiao, Q. Huang, Y. Miao, S. Li, and W. Susilo, "Blockchain based multi-authority fine-grained access control system with flexible revocation," *IEEE Trans. Serv. Comput.*, vol. 15, no. 6, pp. 3143–3155, Nov./Dec. 2022.
[17] J. Yu, S. Liu, M. Xu, H. Guo, F. Zhong, and W. Cheng, "An efficient revocable and searchable MA-ABE scheme with blockchain assistance for C-IoT," *IEEE Internet Things J.*, vol. 10, no. 3, pp. 2754–2766, Feb. 2023.
[18] L. Li, H. Wu, J. Shen, Z. Cao, and X. Dong, "PGVMatch: Privacy-preserving and fine-grained crowdsourcing task matching with lightweight on-chain public verifiability," *IEEE Trans. Mobile Comput.*, vol. 24, no. 9, pp. 8642–8655, Sep. 2025.
[19] C.-I. Fan, S.-J. Wu, Y.-F. Tseng, and A. Karati, "Attribute-based encryption supporting multi-keyword search with effective user revocation in public cloud storage," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 6, pp. 7790–7801, Nov./Dec. 2025.
[20] A. Sahai and B. Waters, "Fuzzy identity-based encryption," in *Proc. Annu. Int. Conf. Theory Appl. Cryptographic Techn.*, 2005, pp. 457–473.
[21] V. Goyal, O. Pandey, A. Sahai, and B. Waters, "Attribute-based encryption for fine-grained access control of encrypted data," in *Proc. ACM Conf. Comput. Commun. Secur.*, 2006, pp. 89–98.
[22] J. Bethencourt, A. Sahai, and B. Waters, "Ciphertext-policy attribute-based encryption," in *Proc. IEEE Symp. Secur. Privacy*, 2007, pp. 321–334.
[23] R. Ostrovsky, A. Sahai, and B. Waters, "Attribute-based encryption with non-monotonic access structures," in *Proc. ACM Conf. Comput. Commun. Secur.*, 2007, pp. 195–203.

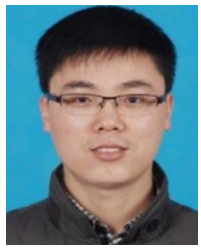
[24] A. Lewko, T. Okamoto, A. Sahai, K. Takashima, and B. Waters, "Fully secure functional encryption: Attribute-based encryption and (hierarchical) inner product encryption," in *Proc. Annu. Int. Conf. Theory Appl. Cryptographic Techn.*, 2010, pp. 62–91.
[25] M. Chase, "Multi-authority attribute based encryption," in *Proc. Theory Cryptogr. Conf.*, 2007, pp. 515–534.
[26] M. Chase and S.S. M. Chow, "Improving privacy and security in multi-authority attribute-based encryption," in *Proc. ACM Conf. Comput. Commun. Secur.*, 2009, pp. 121–130.
[27] J. Han, W. Susilo, Y. Mu, and J. Yan, "Privacy-preserving decentralized key-policy attribute-based encryption," *IEEE Trans. Parallel Distrib. Syst.*, vol. 23, no. 11, pp. 2150–2162, Nov. 2012.
[28] A. Ge, J. Zhang, R. Zhang, C. Ma, and Z. Zhang, "Security analysis of a privacy-preserving decentralized key-policy attribute-based encryption scheme," *IEEE Trans. Parallel Distrib. Syst.*, vol. 24, no. 11, pp. 2319–2321, Nov. 2013.
[29] Y. Rahulamathavan, S. Veluru, J. Han, F. Li, M. Rajarajan, and R. Lu, "User collusion avoidance scheme for privacy-preserving decentralized key-policy attribute-based encryption," *IEEE Trans. Comput.*, vol. 65, no. 9, pp. 2939–2946, Sep. 2016.
[30] R. Longo, C. Marcolla, and M. Sala, "Key-policy multi-authority attribute-based encryption," in *Proc. Int. Conf. Algebr. Informat.*, 2015, pp. 152–164.
[31] D. Ghopur et al., "Decentralized multiauthority attribute-based searchable encryption for E-health cloud," *IEEE Internet Things J.*, vol. 12, no. 11, pp. 15723–15735, Jun. 2025.
[32] A. Beimel, "Secure schemes for secret sharing and key distribution," PhD thesis, Israel Institute of Technology, Technion, 1996.
[33] J. Lai, R. H. Deng, and Y. Li, "Expressive CP-ABE with partially hidden access structures," in *Proc. ACM Symp. Inf., Comput. Commun. Secur.*, 2012, pp. 18–19.
[34] J. A. Akinyele et al., "Charm: A framework for rapidly prototyping cryptosystems," *J. Cryptographic Eng.*, vol. 3, pp. 111–128, 2013.
[35] A. Lewko and B. Waters, "Unbounded hibe and attribute-based encryption," in *Proc. Annu. Int. Conf. Theory Appl. Cryptographic Techn.*, 2011, pp. 547–567.

Chenbin Zhao received the PhD degree in cyberspace security from the School of Cyber Science and Engineering, Wuhan University, Wuhan, China, in 2025. He is currently a lecturer with the Anhui University of Science and Technology, China. He has published more than 20 research papers in international journals and conferences, including *IEEE Transactions on Information Forensics and Security*, *IEEE Transactions on Mobile Computing*, *IEEE Transactions on Dependable and Secure Computing*, TR, and the International Conference on Information and Communication Security, among others. His research interests include applied cryptography and searchable encryption.

Xiaocong Lin received the BS degree from the College of Computer and Cyber Security, Fujian Normal University, China, in 2023. Currently, he is working toward the MS degree with the College of Computer and Cyber Security, Fujian Normal University, China. His research interests include cloud computing security and applied cryptography.

Jing Chen received the PhD degree in computer science from the Huazhong University of Science and Technology, Wuhan. He worked as a full professor with Wuhan University from 2015. His research interests in computer science are in the areas of network security, cloud security. He has published more than 100 research papers in many international journals and conferences, such as *IEEE Transactions on Dependable and Secure Computing*, *IEEE Transactions on Information Forensics and Security*, *IEEE Transactions on Mobile Computing*, *IEEE Transactions on Computers*, *IEEE Transactions on Parallel and Distributed Systems*, USENIX Security, CCS, INFOCOM, et al. He acts as a reviewer for many journals and conferences, such as *IEEE Transactions on Information Forensics*, *IEEE Transactions on Computers*, *IEEE/ACM Transactions on Networking*.

Weijing You received the PhD degree in information security from the University of Chinese Academy of Sciences, China, in 2021. She is currently an associate professor with the College of Computer and Cyber Security, Fujian Normal University, China. She has published several referred research papers in conferences and journals, including NDSS, ESORICS, *IEEE Transactions on Information Forensics and Security*, and *Cybersecurity*. Her research interests include applied cryptography and data security. She is a senior member of China Computer Federation (CCF).



Jie Cui (Senior Member, IEEE) received the PhD degree from the University of Science and Technology of China, Hefei, China, in 2012. He is currently a professor and a PhD supervisor with the School of Computer Science and Technology, Anhui University, Hefei. He has more than 150 scientific publications in reputable journals (e.g., *IEEE Transactions on Dependable and Secure Computing*, *IEEE Transactions on Information Forensics and Security*, *IEEE Journal on Selected Areas in Communications*, *IEEE Transactions on Mobile Computing*, *IEEE Transactions on Parallel and Distributed Systems*, *IEEE Transactions on Computers*, *IEEE Transactions on Vehicular Technology*, *IEEE Transactions on Intelligent Transportation Systems*, *IEEE Transactions on Network and Service Management*, *IEEE Transactions on Industrial Informatics*, *IEEE Transactions on Industrial Electronics*, *IEEE Transactions on Cloud Computing*, and *IEEE Transactions on Multimedia*), academic books, and international conferences. His current research interests include applied cryptography, IoT security, vehicular ad hoc networks, cloud computing security, and software-defined networking (SDN).

Zhongyun Hua (Senior Member, IEEE) received the BS degree in software engineering from Chongqing University, Chongqing, China, in 2011, and the MS and PhD degrees in software engineering from the University of Macau, Macau, China, in 2013 and 2016, respectively. He is currently a professor with the Harbin Institute of Technology, Shenzhen, China. His research has been published in prestigious venues, such as ACM CCS, USENIX Security, *IEEE Transactions on Dependable and Secure Computing*, *IEEE Transactions on Information Forensics and Security*, *IEEE Transactions on Signal Processing*, ICML, and CVPR. His current research interests include applied cryptography, multimedia security, trustworthy AI, nonlinear systems and their applications. He has authored or coauthored more than 130 papers on these topics, which have received more than 9,500 citations. He is an associate editor for several prestigious journals, including *IEEE Transactions on Dependable and Secure Computing* and *IEEE Signal Processing Letters*. He has been recognized as a Highly Cited Researcher from 2022 to 2024.